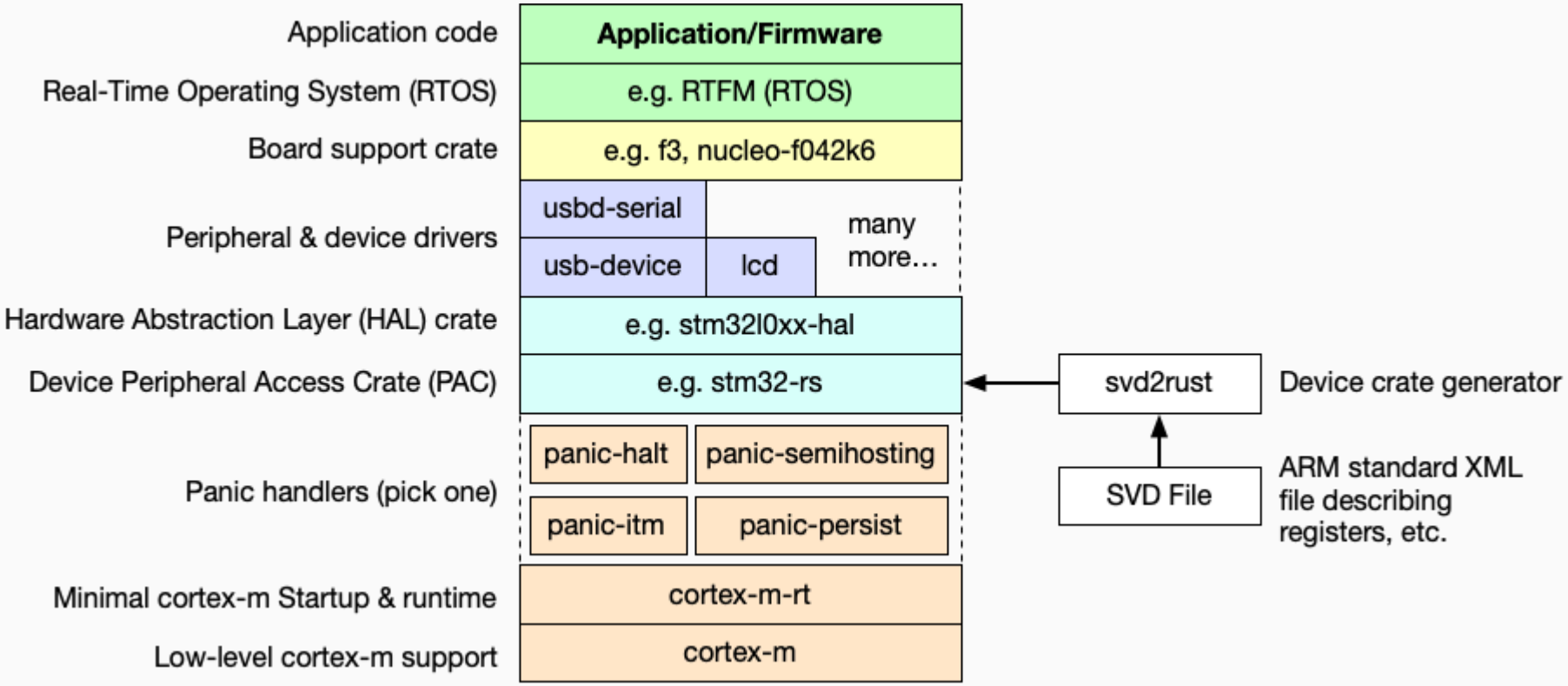
Welcome! 👋

Rust 🦀 has a reputation for being a "low-level" language...

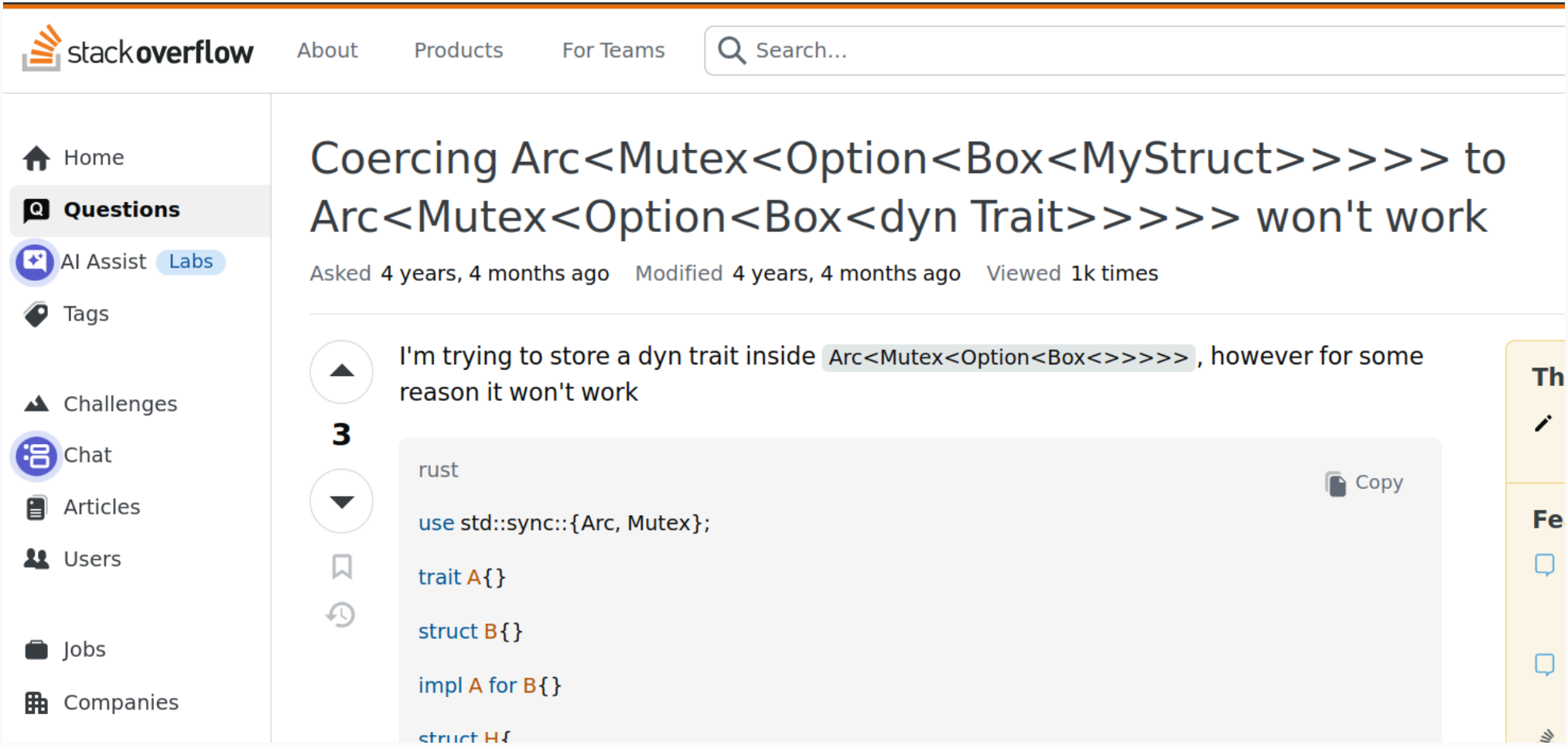


DEVOXX MOROCCO

...for masochists who love memory optimizations...

DEVOXX MOROCCO

...with syntax often called demanding, or even "cryptic"... 🤯



HEY, J'SUIS PAS VENUE ICI POUR
SOUFFRIR, OK ???

Maybe that's all true, but that's not why I'm into Rust

DEVOXX MOROCCO

🦀Rust🦀
for high-level, business apps

An expressive type system

The basics: structs and "newtypes"

A very familiar example:

```rust
pub struct User {
    pub email: String,
}

fn my_function(user: User) {
  if !user.email.contains("@") {
     // Handle the error, etc.
  }
  // Moving on...

  // Allowed, but not what we want -> Bug!
  let city: String = user.email
}
```

Business logic built into types. Living the dream! 😌

```rust
use crate::domain::UserEmail;

pub struct User {
    // Wait... what's that?
    pub email: UserEmail,
}
```

Let's "impl"ement the logic to handle each case:

```rust
use validator::ValidateEmail;

#[derive(Debug)]
pub struct UserEmail(String);

impl UserEmail {
    pub fn parse(s: String) -> Result<UserEmail, String> {
        if s.validate_email() {
            Ok(Self(s))
        } else {
            Err(format!("{} is not a valid user email", s))
        }
    }
}
```

How do we check this? With tests! 🤩

```rust
#[cfg(test)]
mod tests {
    #[test]
    fn email_missing_at_symbol_is_rejected() {
        let email = "stephanedomain.com".to_string();
        assert_err!(UserEmail::parse(email));
    }

    #[test]
    fn email_missing_subject_is_rejected() {
        let email = "@domain.com".to_string();
        assert_err!(UserEmail::parse(email));
    }
}
```

One newtype is not the same as another!

```rust
pub struct Address(String);
pub struct UserEmail(String);

pub struct User { pub email: UserEmail }

let my_user = User {
  // 💥Error! That's a String, not a UserEmail!
  email: "steph@mydomain.com"
}

let my_user = User {
  // 💥Error! That's an Address, not a UserEmail!
  email: Address("steph@mydomain.com")
}
```

Use the right newtype for the job:

```rust
use crate::domain::UserEmail;

pub struct User {
    pub email: UserEmail,
}

let my_user = User {
  // Ok 👍
  email: UserEmail::parse("steph@mydomain.com")?;
}
// Note: for now, `?` is "magic" 🪄
```

ALEXIS KING

Parse, don't validate

2019-11-05 • functional programming, haskell, types

Historically, I've struggled to find a concise, simple way to explain what it means to practice type-driven design. Too often, when someone asks me "How did you come up with this approach?" I find I can't give them a satisfying answer. I know it didn't just come to me in a vision—I have an iterative design process that doesn't require plucking the "right" approach out of thin air—yet I haven't been very successful in communicating that process to others.

However, about a month ago, I was reflecting on Twitter about the differences I experienced parsing JSON in statically- and dynamically-typed languages, and finally, I realized what I was looking for. Now I have a single, snappy slogan that encapsulates what type-driven design means to me, and better yet, it's only three words long:

Parse, don't validate.

https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/

Functional State Machines in Rust: Typestate and Newtype Patterns (Experience Report)

Leon Heuer
Nordakademie
Elmshorn, Germany
leon.heuer.a22b@nordakademie.org

Falk Woldmann Lu
Otto
Hamburg, Germany
research@woldmann.com

Jan Haase
Nordakademie
Elmshorn, Germany
jan.haase@nordakademie.de

Abstract

We investigate the effects on code quality of the design patterns typestate and newtype in the language Rust, by conducting three case studies from production code, evaluated through expert interviews, static code analysis and benchmarks. The findings indicate that: (1) Typestate improves code faultlessness and testability, but comes at the cost of more boilerplate code and can degrade readability. Code with extensive branching logic and many invariants likely benefits most from the pattern. (2) Newtype combined with the "Parse, don't validate" principle improves code quality at a low cost and prevents invalid states at runtime.

CCS Concepts: • **Software and its engineering** → *Patterns.*

Keywords: Rust, design patterns, typestate, newtype, functional core imperative shell, code quality, experience report

ACM Reference Format:

Most catalogued design patterns as of now originate in OOP, for example the Gang of Four (GoF) patterns [6]. Many GoF patterns become obsolete or unimplementable when transferred to Rust, since it is more expressive and provides native constructs for the problems those patterns solve.

Rust-specific patterns such as typestate and newtype have emerged. The Rust open source community documents a few of them in rust-unofficial/patterns. Crichton proposes four recurring functional design patterns in Rust and demonstrates their benefits through constructed case studies [4]. These resources provide useful practical guidance but lack evidence on the patterns' impact on code quality.

5 Conclusion

We conducted a case study on three use cases from Otto's backend applications, guided by a method built on SQuaRE criteria and combining execution-time benchmarks, static code analysis and expert interviews.

Neither typestate nor newtype incurred additional runtime cost. Typestate clearly improved testability and faultlessness by shifting error detection to compile time. However, its effect on readability is contested and is not captured by the static metrics. The additional boilerplate is perceived as cognitive overhead by some experts, while others find the explicit states easier to follow. Combined with the principle "Parse, don't validate", the newtype pattern introduced minimal downsides and code changes while improving almost all quality dimensions, making it broadly applicable.

Beyond addressing the limitations from subsection 4.4, several research directions open up. Because typestate's main downside is boilerplate, future studies should investigate macro rules or libraries that generate the state scaffolding and let developers focus on the logic of each state transition, similar to Duarte and Ravara [5]. Future work could furthermore strengthen the link between business requirements and source code. Since encoding invariants already transfers

Impureim sandwich by Mark Seemann

Pronounced 'impurium sandwich'.

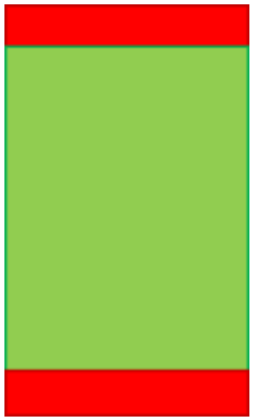
Since January 2017 I've been singing the praise of the *impure/pure/impure* sandwich, but I've never published an article that defines the term. I intend this article to remedy the situation.

Functional architecture

In a functional architecture, pure functions can't call impure actions. On the other hand, as Simon Peyton Jones observed in a lecture, *observing the result of pure computation is a side-effect*. In practical terms, *executing* a pure function is also impure, because it happens non-deterministically. Thus, even for a piece of software written in a functional style, the entry point must be impure.

While pure functions can't call impure actions, there's no rule to prevent the obverse. Impure actions *can* call pure functions.

Therefore, the best we can ever hope to achieve is an impure entry point that calls pure code and impurely reports the result from the pure function.



The flow of code here goes from top to bottom:

1. Gather data from impure sources.
2. Call a pure function with that data.
3. Change state (including user interface) based on return value from pure function.

This is the *impure/pure/impure* sandwich.

The A and Ω of Type-Driven Development…

Pattern Matching

Let's take a simple enum:

```
pub enum LoginError {
    AuthenticationError(String),
    SSOErrorCode(u32),
    UnexpectedError(String),
}
```

The `match` operator

```
fn my_function(e: LoginError) {
    match e {}
}
```

```
+-----------------------------------------------------------------------------+
|missing match arm: `AuthenticationError(_)`, `SSOErrorCode(_)` and           |
|`UnexpectedError(_)` not covered (rust-analyzer E0004)                       |
|-----------------------------------------------------------------------------|
|non-exhaustive patterns: `LoginError::AuthenticationError(_)`, `LoginError:: |
|SSOErrorCode(_)` and `LoginError::UnexpectedError(_)` not covered            |
|the matched value is of type `LoginError`                                    |
|                                                                             |
|Related information:                                                         |
|                                                                             |
|  * main.rs#463,10: `LoginError` defined here                                |
|  * main.rs#470,12: ensure that all possible cases are being handled by      |
|adding a match arm with a wildcard pattern, a match arm with multiple or-    |
|patterns as shown, or multiple match arms: ` {                              |
|        LoginError::AuthenticationError(_) | LoginError::SSOErrorCode(_) |   |
|LoginError::UnexpectedError(_) => todo!(),                                   |
|    }`                                                                       |
|                                                                             |
| (rustc E0004)                                                               |
|-----------------------------------------------------------------------------|
|https://doc.rust-lang.org/error-index.html#E0004                             |
+-----------------------------------------------------------------------------+
```

Luckily, Rust's language server has our back 😌

```
fn my_function(e: LoginError) {
    match e {}
}           +Choose action------+
            |1. Fill match arms |
            +-------------------+
```

Just fill in the blanks!

```
fn my_function(e: LoginError) {
    match e {
        LoginError::AuthenticationError(_) => todo!(),
        LoginError::SSOErrorCode(_) => todo!(),
        LoginError::UnexpectedError(_) => todo!(),
    }
}
```

We can leave the real implementation for later

```
fn my_function(e: LoginError) {
    match e {
        LoginError::AuthenticationError(_) => todo!("Oh, not cool 😱"),
        LoginError::SSOErrorCode(_) => todo!("Ouch! Even less cool 😓"),
        LoginError::UnexpectedError(_) => todo!("I can't take this anymore… 😅"),
    }
}
```

Matching can be as complex as we need

```
fn my_function(e: LoginError) {
    match e {
        LoginError::AuthenticationError(auth)
          if auth.contains("network") => todo!("Oh, not cool 😱")
        LoginError::SSOErrorCode(c)
          if c > 42 && c < 1337 => todo!("Ouch! Even less cool 😓"),
        LoginError::UnexpectedError(error)
          if error.is_empty() => todo!("I can't take this anymore… 😅"),
    }
}
```

But the compiler is watching...

```
fn my_function(e: LoginError) {
    match e {
          +-------------------------------------------------------------------------------+
        Lo|missing match arm: `AuthenticationError(_)` not covered (rust-analyzer E0004) |
          |-------------------------------------------------------------------------------|
        } |non-exhaustive patterns: `LoginError::AuthenticationError(_)` not covered      |
        Lo|the matched value is of type `LoginError`                                      |
        Lo|                                                                               |
    }     |Related information:                                                           |
}         |                                                                               |
          |  * main.rs#463,10: `LoginError` defined here                                  |
          |  * main.rs#475,76: ensure that all possible cases are being handled by       |
          |adding a match arm with a wildcard pattern or an explicit pattern as shown: `,|
          |        LoginError::AuthenticationError(_) => todo!()`                         |
          |                                                                               |
          | (rustc E0004)                                                                 |
          |-------------------------------------------------------------------------------|
          |https://doc.rust-lang.org/error-index.html#E0004                               |
          +-------------------------------------------------------------------------------+
```

Can you spot the problem here?

```
pub enum Direction {
    North,
    South,
    East,
    West,
}
type Coordinates<'a> = (&'a u32, &'a u32);

pub fn move_player((&x, &y): Coordinates, direction: &Direction) {
    match ((x, y), direction) {
        ((0, 0), _) => todo!("At the origin, nothing to do"),
        ((13, 10), Direction::South) => todo!("Nope, there's an ocean that way!"),
    }
}
```

The compiler pays more attention than you do 😸

```
pub fn move_player((&x: u32, &y: u32): Coordinates, direction: &Direction) {
    match ((x, y), direction) {
          +-------------------------------------------------------------------------------+
        (((|non-exhaustive patterns: `((1_u32..=12_u32, _), _)` and `((14_u32..=u32::MAX,|
        (((|_), _)` not covered                                                          |
    }     |the matched value is of type `((u32, u32), &Direction)`                     |
}         |                                                                             |
          |Related information:                                                         |
          |                                                                             |
          |  * main.rs#412,77: ensure that all possible cases are being handled by      |
          |adding a match arm with a wildcard pattern, a match arm with multiple or-    |
          |patterns as shown, or multiple match arms: `,                                |
          |        ((1_u32..=12_u32, _), _) | ((14_u32..=u32::MAX, _), _) => todo!()`   |
          |                                                                             |
          | (rustc E0004)                                                               |
          |-----------------------------------------------------------------------------|
          |https://doc.rust-lang.org/error-index.html#E0004                             |
          +-------------------------------------------------------------------------------+
```

And in the end, it all works out 😉

```
pub enum Direction {
    North,
    South,
    East,
    West,
}
type Coordinates<'a> = (&'a u32, &'a u32);

pub fn move_player((&x, &y): Coordinates, direction: &Direction) {
    match ((x, y), direction) {
        ((0, 0), _) => todo!("At the origin, nothing to do"),
        ((13, 10), Direction::South) => todo!("Nope, there's an ocean that way!"),
        ((_,_) , _) => todo!("Handle the other cases later")
    }
}
```

Rust's "Boxes"

DEVOXX MOROCCO

Kahoot time!

DEVOXX MOROCCO

Yep, it's Null:

Speaking at a software conference in 2009, Tony Hoare hyperbolically apologized for inventing the null reference: [26][27]

> I call it my billion-dollar mistake. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object oriented language (ALGOL W). My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.[28]

In JavaScript (and others...):

DEVOXX MOROCCO

In Rust, no more guessing!

```
// get_user_by_email:
//
//   UserEmail -> Option<User>
//
match get_user_by_email(user_email).await {

    Some(user) => todo!("handle the case where the user exists"),

    None => todo!("create the user? return an error? you decide!"),

}
```

Kahoot (yes, again !)

DEVOXX MOROCCO

Why Do Some Developers Consider `Try-Catch` to Be Problematic?

The sentiment that `try-catch` is "evil" stems from several potential pitfalls:

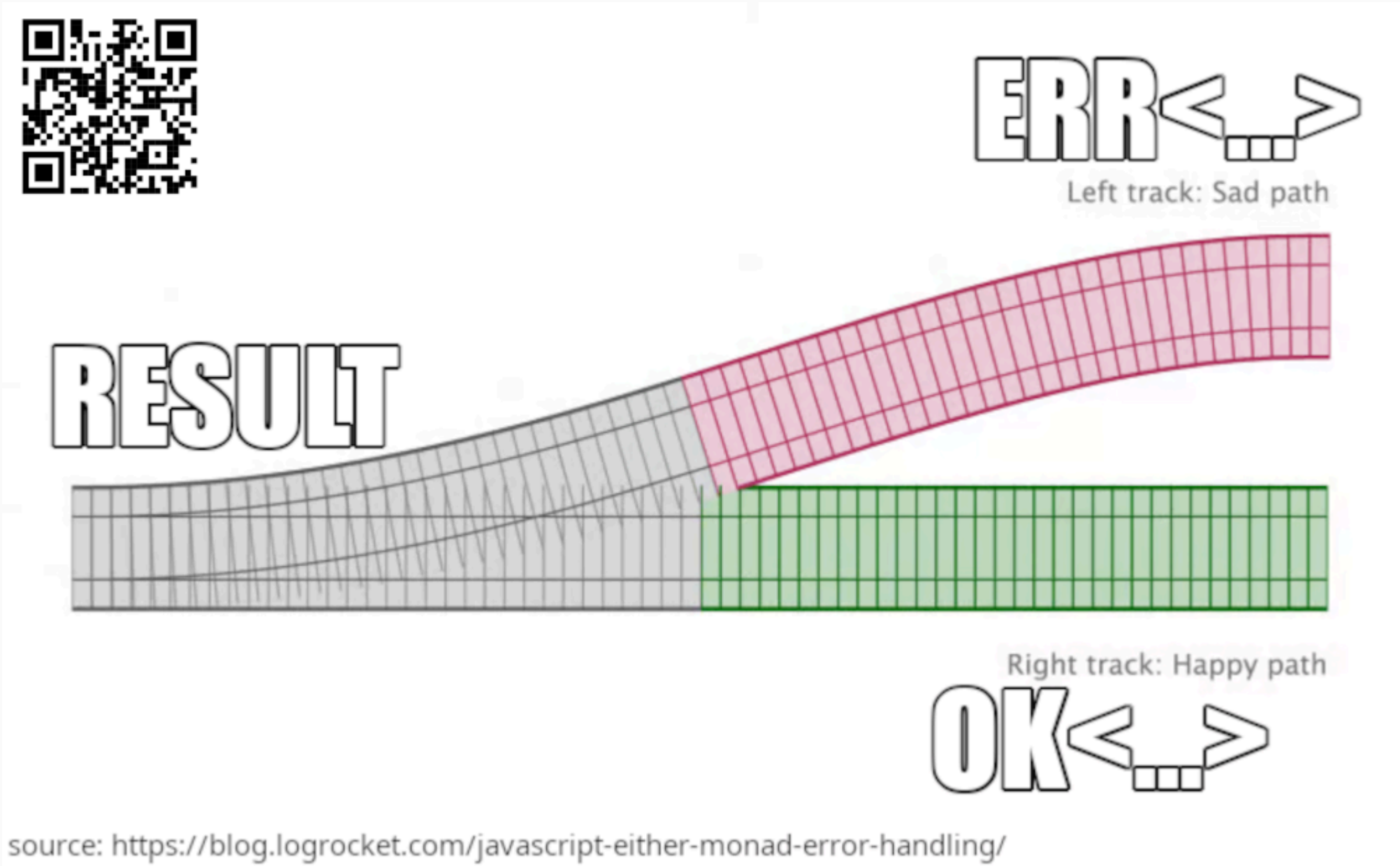
- **Masking Errors**: Catching exceptions too broadly can hide underlying problems, making debugging difficult.
- **Performance Overhead**: Exception handling can be computationally expensive, especially if exceptions are frequently thrown and caught.
- **Code Complexity**: Excessive `try-catch` blocks can clutter code and make it harder to read and maintain.
- **Control Flow**: Using exceptions for normal control flow is generally bad practice and can lead to unexpected behavior.

https://deepdatawithmivaa.com/why-is-try-catch-evil

Some languages do their best… 😅

DEVOXX MOROCCO

Others, like Rust, dive into the multiverse! 💪



Remember `UserEmail`? 🤓

```rust
use validator::ValidateEmail;

#[derive(Debug)]
pub struct UserEmail(String);

impl UserEmail {                +-------------------------+
    pub fn parse(s: String) -> |Result<UserEmail, String>| {
                                +-------------------------+
        if s.validate_email() {
           +-----------+
           |Ok(Self(s))|
           +-----------+
        } else {
           +-----------------------------------------------+
           |Err(format!("{} is not a valid user email", s))|
           +-----------------------------------------------+
        }
    }
}
```

And since Result is an enum, we can `match` it 🤩

```
// validate_credentials:
//
//   Credentials -> Result<Uuid, LoginError>
//
match validate_credentials(credentials).await {
}                       +Choose action -----+
                        |1. Fill match arms |
                        +-------------------+
```

Just fill in the blanks, or slap a `todo!` on it 🤷

```
// validate_credentials:
//
//   Credentials -> Result<Uuid, LoginError>
//
match validate_credentials(credentials).await {

    Ok(_) => todo!("handle the case where everything is OK"),

    Err(_) => todo!("add error handling"),

}
```

Finally, we add the real code:

```
match validate_credentials(credentials).await {
  Ok(user_id) => {
    return HttpResponse::SeeOther()
      .insert_header((LOCATION, "/admin/dashboard"))
      .finish());
  }
  Err(e) => {
    return login_redirect(LoginError::AuthError(e));
  }
}
```

"Do we have to `match` every Result?" 🤔

No, we can use `.unwrap()`, among other things:

```
// Here, we're convinced we'll
// always have a port available
let port = listener.local_addr().unwrap().port();
```

So why put ourselves through `match`? ☝️🤓

DEVOXX MOROCCO

...even the big players get caught out 💥

When the bad file with more than 200 features was propagated to our servers, this limit was hit — resulting in the system panicking. The FL2 Rust code that makes the check and was the source of the unhandled error is shown below:

```
71      /// Fetch edge features based on `input` struct into [`Features`] buffer.
72      pub fn fetch_features(
73          &mut self,
74          input: &dyn BotsInput,
75          features: &mut Features,
76      ) -> Result<(), (ErrorFlags, i32)> {
77          // update features checksum (lower 32 bits) and copy edge feature names
78          features.checksum &= 0xFFFF_FFFF_0000_0000;
79          features.checksum |= u64::from(self.config.checksum);
80          let (feature_values, _) = features
81              .append_with_names(&self.config.feature_names)
82              .unwrap();
```

This resulted in the following panic which in turn resulted in a 5xx error:

```
thread fl2_worker_thread panicked: called Result::unwrap() on an Err value
```

https://blog.cloudflare.com/18-november-2025-outage/

Save `.unwrap()` for tests, and learn to propagate or handle `Result` properly

```
fn my_function() -> Result<PortNumber, Error> {
  /* In this case, we SHOULD have an IP address. */
  /* If not, bubble up the error upwards */
  let port = listener.local_addr()?.port();
  return port;
}
```

Remember: Clippy is your friend

unwrap_used

restriction | allow | –

What it does

Checks for `.unwrap()` or `.unwrap_err()` calls on `Results` and `.unwrap()` call on `Options`.

Why restrict this?

It is better to handle the `None` or `Err` case, or at least call `.expect(_)` with a more helpful message. Still, for a lot of quick-and-dirty code, `unwrap` is a good choice, which is why this lint is `Allow` by default.

`result.unwrap()` will let the thread panic on `Err` values. Normally, you want to implement more sophisticated error handling, and propagate errors upwards with `?` operator.

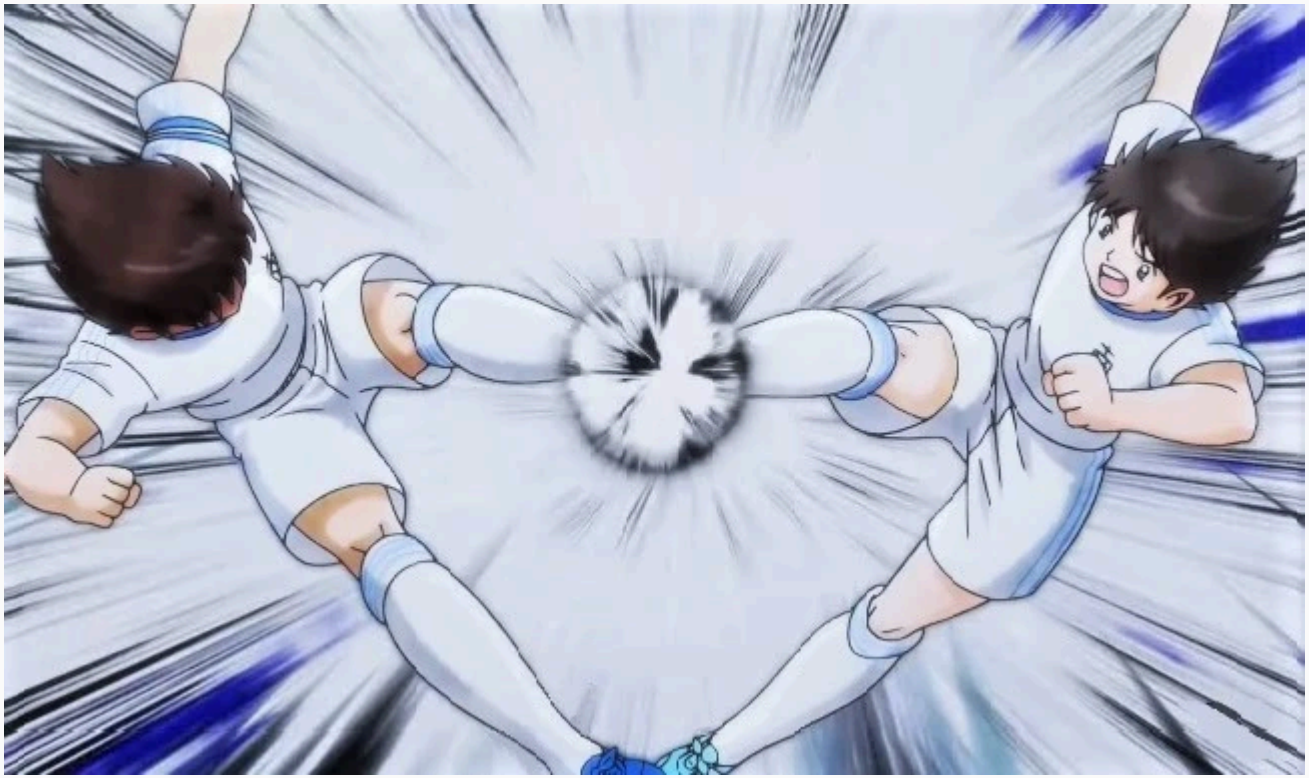
Even if you want to panic on errors, not all `Errors` implement good messages on display. Therefore, it may be beneficial to look at the places where they may get displayed. Activate this lint to do just that.

DEVOXX MOROCCO

Speaking of errors...

The wonder twins of error handling

anyhow and thiserror



DEVOXX MOROCCO

anyhow: errors made easy

```rust
use anyhow::{Context, Result};

/*  Result doesn't even need an error type here */
/*  because it will be an `anyhow::Error` 👌 */
fn main() -> Result<()> {

  /*  .context wraps the error with… context 😁 */
  it.detach().context("Failed to detach the thing")?;

  /*  Using `?` simply means: */
  /*  If it's an `Err`, bubble it up! */
  /*  Otherwise, "open" the `Ok` box and take what's inside */
  let content = std::fs::read(path)?;
}
```

`thiserror`: errors made easy to describe

```rust
use thiserror::Error;
#[derive(Error, Debug)]
pub enum DataStoreError {
  #[error("data store disconnected")]
  Disconnect(#[from] io::Error),
  #[error("the data for key `{0}` is not available")]
  Redaction(String),
  #[error("invalid ({expected:?}, found {found:?})")]
  InvalidHeader {
      expected: String,
      found: String,
  },
  #[error("unknown data store error")]
  Unknown
}
```

`thiserror` for all types? `derive_more`!

```rust
use derive_more::{Add, Display, From, Into};

#[derive(PartialEq, From, Add)]
struct MyInt(i32);

#[derive(PartialEq, From, Into)]
struct Point2D {
    x: i32,
    y: i32,
}

#[derive(PartialEq, From, Add, Display)]
enum MyEnum {
    #[display("int: {_0}")]
    Int(i32),
    Uint(u32),
    #[display("nothing")]
    Nothing,
}
```

Declaring an enum feels like too much work?
eros to the rescue!

```rust
use eros::IntoUnion;
use std::{io, num::ParseIntError};

fn load_port(path: &str) -> eros::Result<u16, (io::Error, ParseIntError)> {
    let contents = std::fs::read_to_string(path).union()?;
    contents.trim().parse().union()
}

fn port_or_default(path: &str) -> eros::Result<u16, (ParseIntError,)> {
    load_port(path).recover::<io::Error, _>(|error| {
        eprintln!("{error:?}; using port 8080");
        8080
    })
}
```

Traits: turbocharged OOP

We define what a `PaymentProcessor` is

```rust
pub struct RecipientEmail(pub String);
pub struct SenderEmail(pub String);

pub trait PaymentProcessor {
    fn send_payment(
        recipient: &RecipientEmail,
        sender: &SenderEmail,
    ) -> Result<(), anyhow::Error>;
}
```

Then we implement "our own" `PaymentProcessor`

```rust
pub struct RecipientEmail(pub String);
pub struct SenderEmail(pub String);

pub trait PaymentProcessor {
    fn send_payment(
        recipient: &RecipientEmail,
        sender: &SenderEmail,
    ) -> Result<(), anyhow::Error>;
}

pub struct MyPaymentProcessor {}

impl PaymentProcessor for MyPaymentProcessor {}
```

```rust
pub struct RecipientEmail(pub String);
pub struct SenderEmail(pub String);

pub trait PaymentProcessor {
    fn send_payment(
        recipient: &RecipientEmail,
        sender: &SenderEmail,
    ) -> Result<(), anyhow::Error>;
}

pub struct MyPaymentProcessor {}

impl PaymentProcessor for MyPaymentProcessor {}
```

```
   +-----------------------------------------------------------------------------+
   |not all trait items implemented, missing: `send_payment`                    |
   |missing `send_payment` in implementation                                     |
   |                                                                             |
   |Related information:                                                         |
   |                                                                             |
   |  * main.rs#422,5: `send_payment` from trait                                 |
   |  * main.rs#431,47: implement the missing item: `fn send_payment(_:          |
   |&RecipientEmail, _: &SenderEmail) -> std::result::Result<(), anyhow::Error> { |
   |todo!() }`: `fn send_payment(_: &RecipientEmail, _: &SenderEmail) ->         |
   |std::result::Result<(), anyhow::Error> { todo!() }`                          |
   +-----------------------------------------------------------------------------+
```

Just like with enums, Rust's powerful language server can help

```
pub struct MyPaymentProcessor {}

impl PaymentProcessor for MyPaymentProcessor {}
      +Choose action --------------+
      |1. Implement missing members|
      +----------------------------+
```

Then we add whatever implementation we want:

```rust
pub struct RecipientEmail(pub String);
pub struct SenderEmail(pub String);

pub trait PaymentProcessor {
    fn send_payment(
        recipient: &RecipientEmail,
        sender: &SenderEmail,
    ) -> Result<(), anyhow::Error>;
}

pub struct MyPaymentProcessor {}
impl PaymentProcessor for MyPaymentProcessor {
    fn send_payment(
        recipient: &RecipientEmail,
        sender: &SenderEmail,
    ) -> Result<(), anyhow::Error> {
        todo!("Payment instructions from {sender} to {recipient} go here !")
    }
}
```

The "standard" traits

A simple example: I have a `LoginError` and I want to debug it

```
pub enum LoginError {
    AuthenticationError(String),
    UnexpectedError(String),
}
```

A super handy "function": dbg !

```
let my_error = LoginError::AuthError("Uh-oh!");
dbg!(&my_error)
```

Except LoginError isn't a primitive type...

```
+-------------------------------------------------------+
|`LoginError` doesn't implement `std::fmt::Debug`       |
|add `#[derive(Debug)]` to `LoginError` or manually `impl |
|std::fmt::Debug for LoginError`                        |
|                                                       |
|Related information:                                   |
|                                                       |
|  * error.rs#53,18: required by a bound in `Error`     |
|  * post.rs#18,1: consider annotating `LoginError` with  |
|    `#[derive(Debug)]`                                 |
|                                                       |
| (rustc E0277)                                         |
|-------------------------------------------------------|
|https://doc.rust-lang.org/error-index.html#E0277       |
+-------------------------------------------------------+
```

Deriving: simple and handy in 90% of cases

```
#[derive(Debug)]
pub enum LoginError {
    AuthError(Error),
    UnexpectedError(Error),
}
```

Fine for simple cases, but how do we implement Debug ourselves?

Once again, the language server is here to help 😍

```
pub enum LoginError {
    AuthError(String),
    UnexpectedError(String),
}

impl std::fmt::Debug for LoginError {}
              +Choose action --------------+
              |1. Implement missing members|
              +----------------------------+
```

We get a "default" implementation we can customize 🤤

```rust
impl std::fmt::Debug for LoginError {
  fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    match self {
      Self::AuthError(arg0) =>
          f.debug_tuple("AuthError").field(arg0).finish(),
      Self::UnexpectedError(arg0) =>
          f.debug_tuple("UnexpectedError").field(arg0).finish(),
    }
  }
}
```

We can put anything in it (as long as we follow the trait)

```
impl std::fmt::Debug for LoginError {
    fn fmt(&self, f: &mut Formatter<'_>) -> Result {
        writeln!(f, "{}\n", e)?;
        let mut current = e.source();
        while let Some(cause) = current {
            writeln!(f, "Caused by:\n\t{}", cause)?;
            current = cause.source();
        }
        Ok(())
    }
}
```

Now we get much clearer error details when debugging:

```
Failed to log in user.
Caused by:
    Failed to retrieve user informations
Caused by:
    error returned from database
Caused by:
    table "user" does not exist
```

A "blanket implementation" of one trait for all COMPATIBLE types

DEVOXX MOROCCO

Idea: I want to customize how my variables are logged 🤔

```rust
trait Loggable { fn log(&self); }

pub struct Cat { pub name: String, }

fn main() {
    let numbers = vec![1, 2, 3];
    numbers.log(); // Error! 💥
    // Oh, right! I need to implement Loggable for u64 😫

    let s = String::from("Comfy blanket");
    s.log(); // Error! 💥
    // Oh, right! I need to implement Loggable for String 😫

    let c = Cat { name: "Mimouchka".to_string(), };
    c.log(); // Error! 💥
    // Oh, right! I need to implement Loggable for Cat 😫
}
```

`impl<T>`, because we're worth it 💇

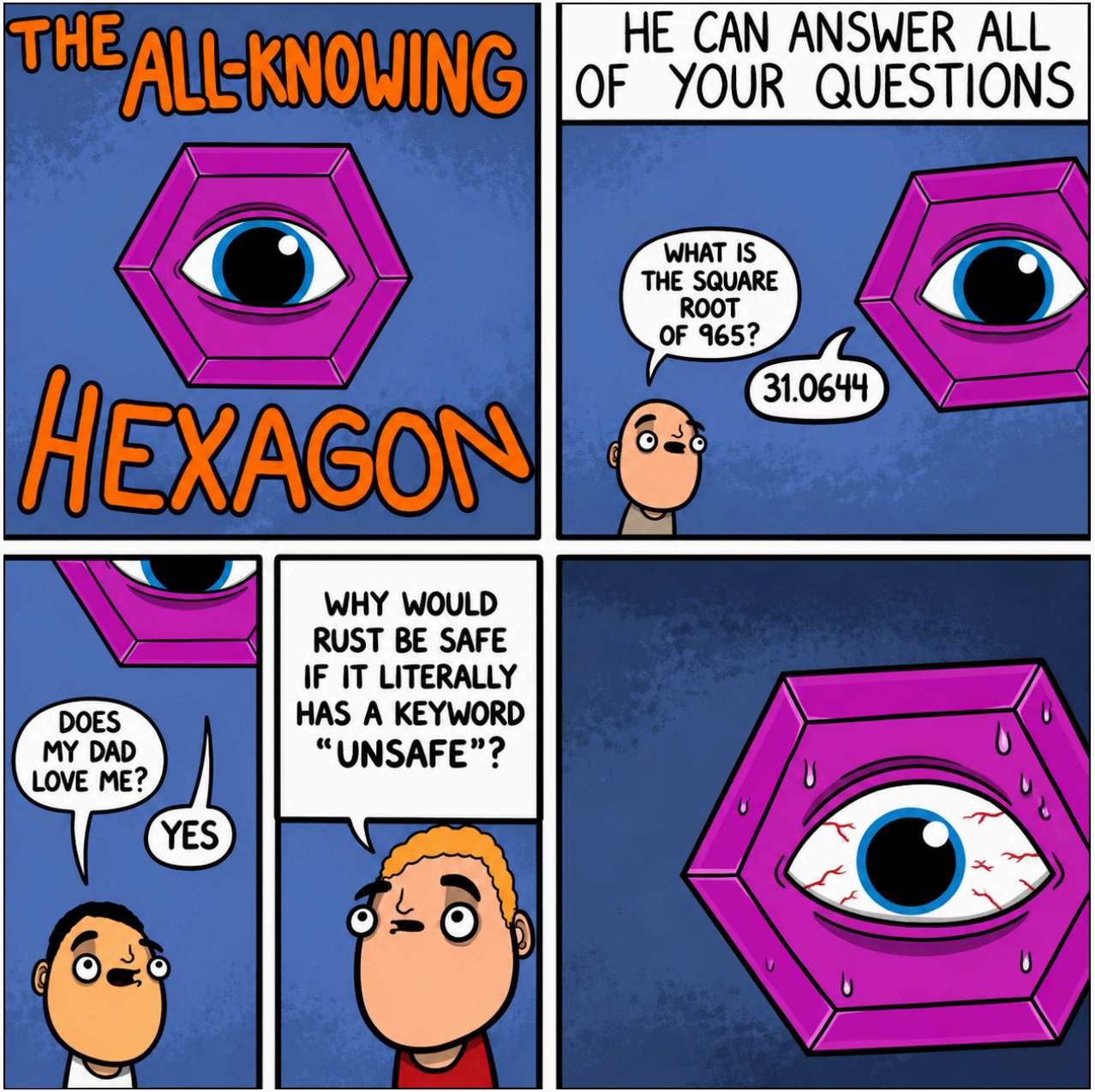
```rust
trait Loggable { fn log(&self); }

// Anything that implements Debug now implements Loggable automatically! 🤩
impl<T> Loggable for T
where T: Debug {
    fn log(&self) { println!("[log] {:?}", self); }
}

#[derive(Debug)]
pub struct Cat { pub name: String, }

fn main() {
    let numbers = vec![1, 2, 3];
    numbers.log(); // Vec<i32> implements Debug out of the box, so Loggable too! 😌

    let s = String::from("Comfy blanket");
    s.log(); // String implements Debug out of the box, so Loggable too! 😌

    let c = Cat { name: "Astra".to_string(), };
    c.log(); // Cat implements Debug (via derive), so Loggable too! 😌
}
```

DEVOXX MOROCCO

And Rust has TONS of types, traits and impls like these 🤩

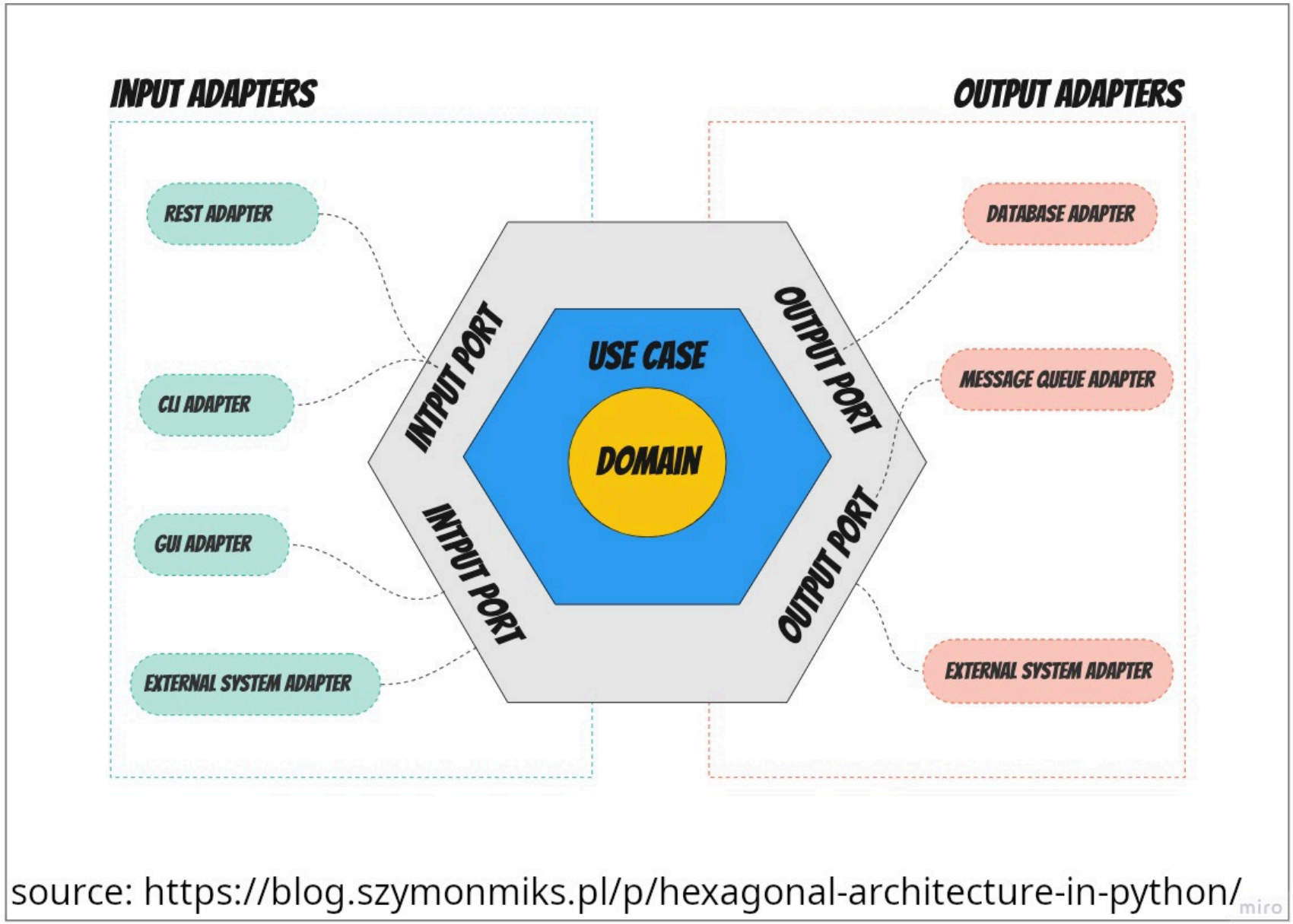
```
- `Option` (`Some<…>` or `None`)
- `Debug` vs `Display`
- `Deref`
- `Drop`
- ...and many more (plus all the community ones 🤍)
```

Hexagonal Architecture



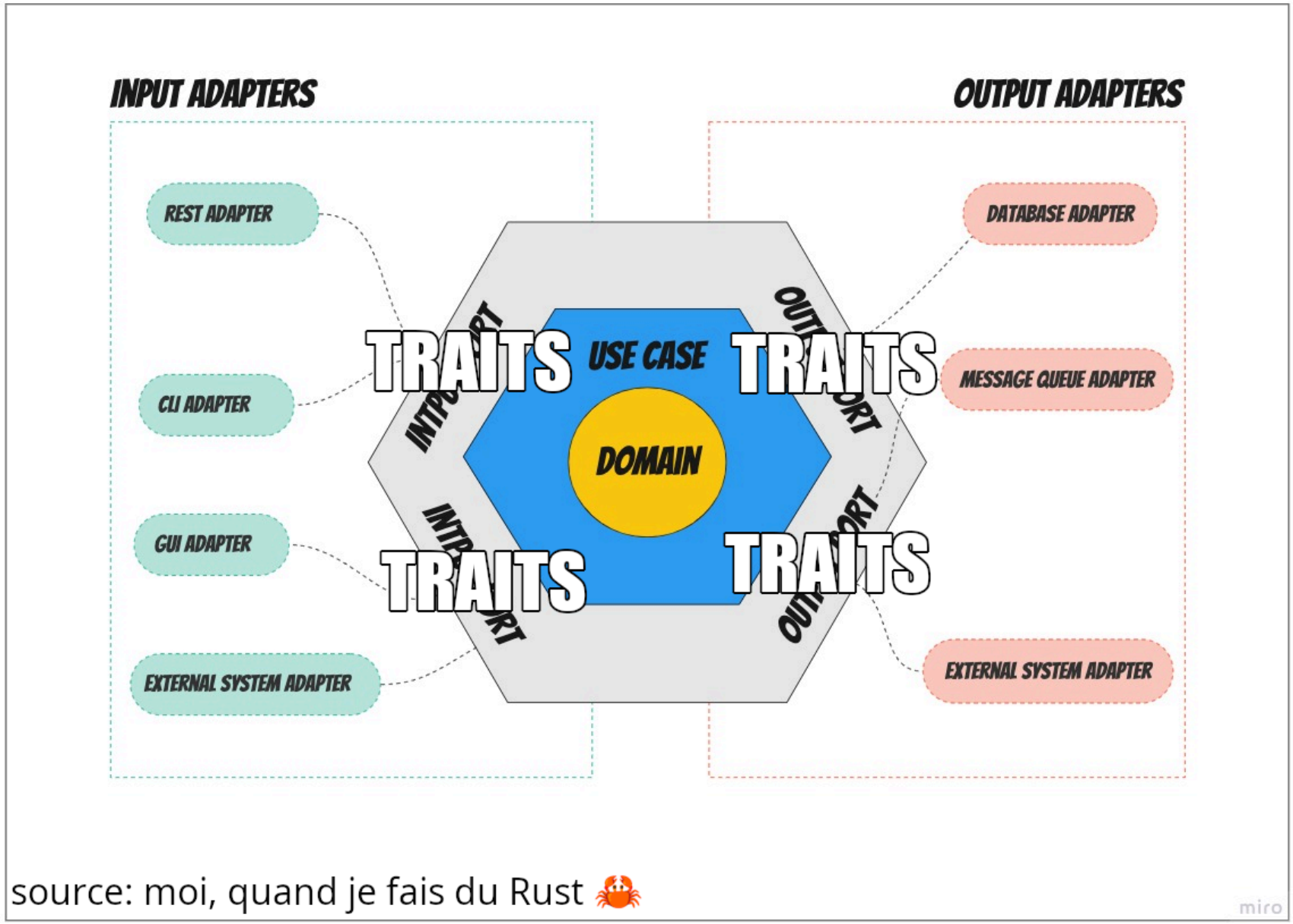
DEVOXX MOROCCO

A quick reminder:



DEVOXX MOROCCO

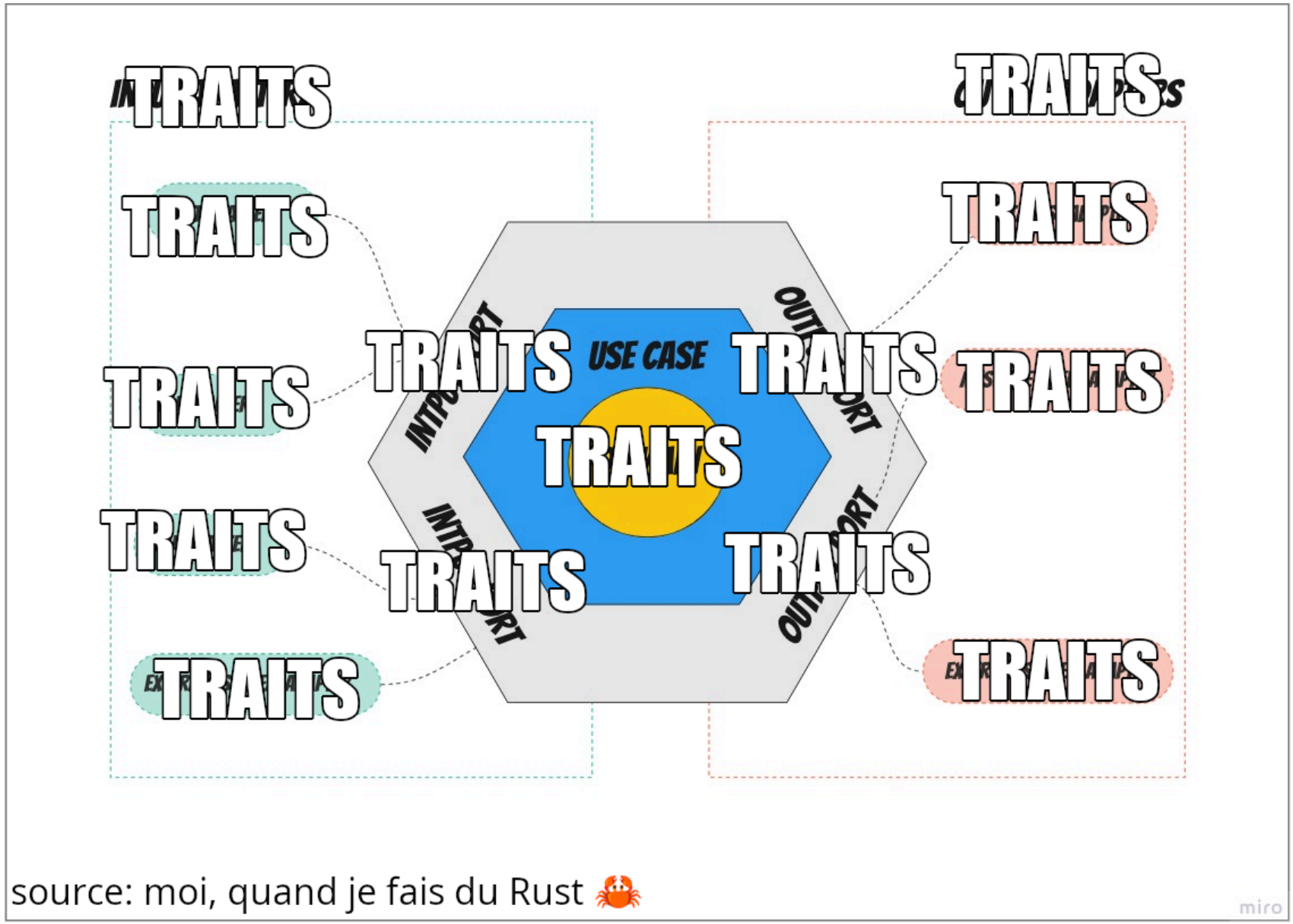
So, if you've been following along…



source: moi, quand je fais du Rust 🦀

DEVOXX MOROCCO

But, in real life…



DEVOXX MOROCCO

And that's OK!

```rust
fn process_data<F, B>(foo: F, bar: B) -> String
where
    F: Clone + Debug,
    B: Display + PartialEq,
{
    // Function implementation
    format!("{:?} - {}", foo, bar)
}
```

IDIOMATIC RUST

THINKING IN TRAITS

by Matthias Endler Published: 2026-11-02

Rust favors composition over inheritance. I'm sure you've heard that before, but rarely does anyone take the time to explain what it means in practice. Also, why is this model the right choice for Rust?

Let's take a step back and look at an example: have you ever asked yourself why `collect()` works the way it does?

```
let doubled: Vec<_> = numbers.iter().map(|x| x * 2).collect();
```

How does `collect` know to produce a `Vec` here? To answer that, let's first look at the signature of `Iterator::collect`:

```
fn collect<B: FromIterator<Self::Item>>(self) -> B
where
    Self: Sized,
{
    FromIterator::from_iter(self)
}
```

Note how it doesn't mention `Vec` at all.

⬢ + 🦀 = 💓

```rust
pub async fn create_author<AR: AuthorRepository>(
    State(state): State<AppState<AR>>,
    Json(body): Json<CreateAuthorHttpRequestBody>,
) -> Result<ApiSuccess<CreateAuthorResponseData>, ApiError> {
    let domain_req = body.try_into_domain()?;
    state
        .author_repo
        .create_author(&domain_req)
        .await
        .map_err(ApiError::from)
        .map(|ref author| ApiSuccess::new(StatusCode::CREATED, author.into()))
}
```

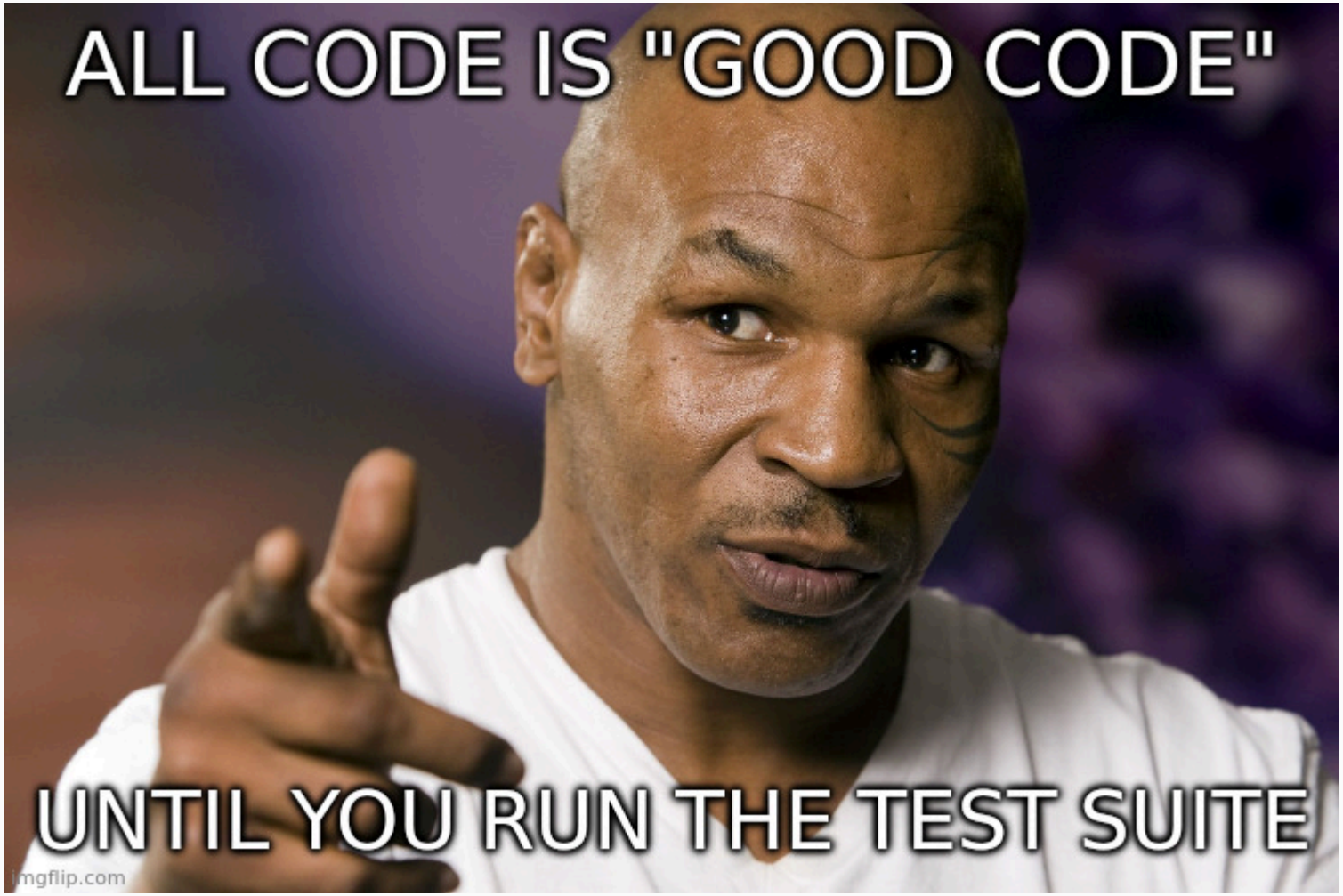
source: https://www.howtocodeit.com/guides/master-hexagonal-architecture-in-rust

Oh my.

Isn't it beautiful?

Doesn't your nervous system feel calmer to behold it?

Testing in Rust



DEVOXX MOROCCO

Docs and tests live right next to the code 💕

```
…/temp-rust  trunk  v1.93.0  ?
(ins)› cargo test -q

running 2 tests
. 1/2
tests::test_adder_ko ── FAILED

failures:

─── tests::test_adder_ko stdout ───

thread 'tests::test_adder_ko' (186854) panicked at src/lib.rs:23:9:
assertion `left == right` failed
  left: 3
 right: 4
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    tests::test_adder_ko

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.0
0s

error: test failed, to rerun pass `--lib`
```

```rust
/// Adder: Add two numbers.
/// ```
/// let x = 5;
/// let y = 5;
/// assert_eq!(mylib::adder(x,y), 10);
/// ```
pub fn adder<T>(x: T, y: T) -> T
where T: std::ops::Add<Output = T> {
    x + y
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_adder_ok() {
        assert_eq!(adder(1, 2), 3);
    }

    #[test]
    fn test_adder_ko() {
        assert_eq!(adder(1, 2), 4);
    }
}
```

Rust project docs are… *chef's kiss* 👌

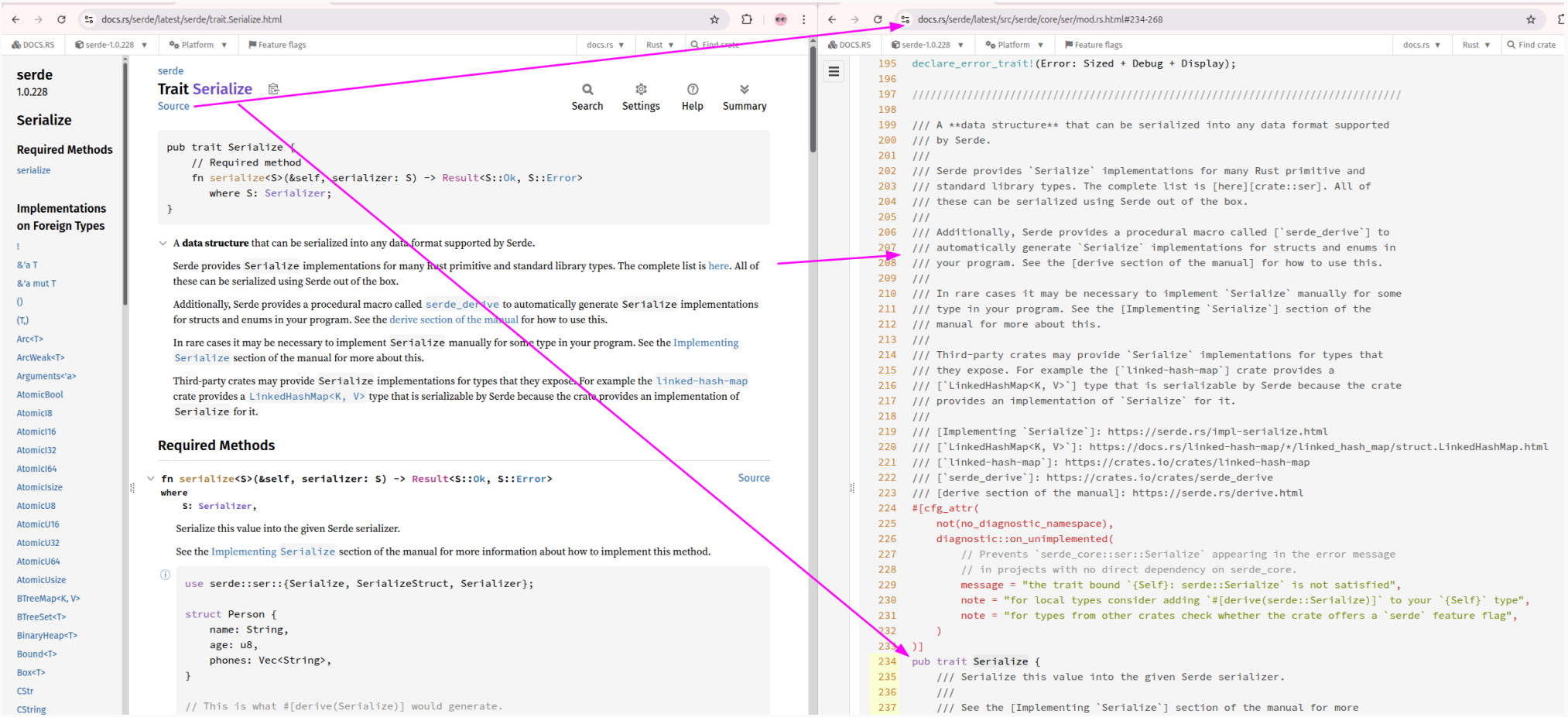
```rust
use mylib::adder;

fn main() {
println!("{}", adder(40, 2));
}
```

```rust
mylib

pub fn adder<T>(x: T, y: T) -> T
where
    T: std::ops::Add<Output = T>,
```

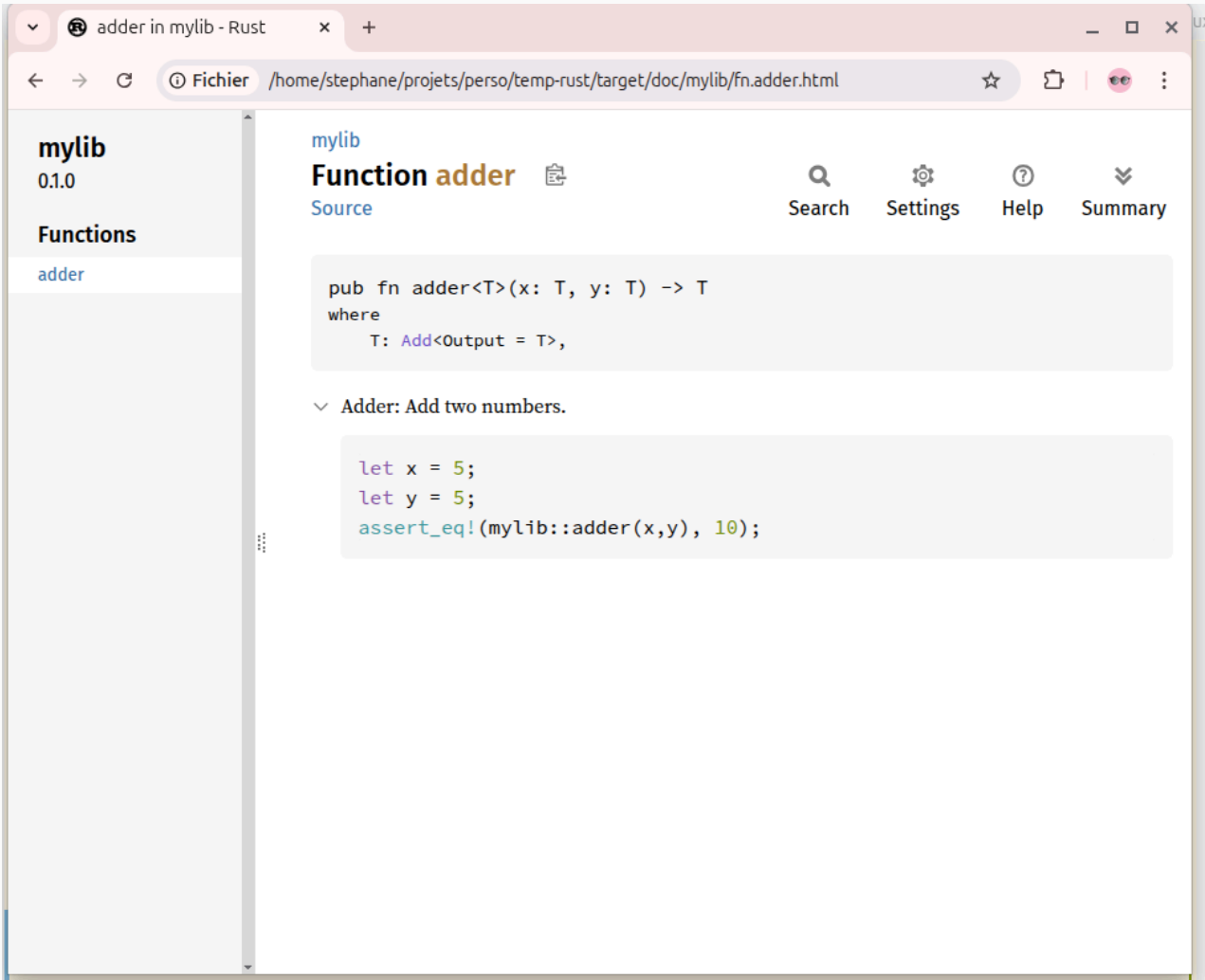
```
T = i32
```

```rust
Adder: Add two numbers.
let x = 5;
let y = 5;
assert_eq!(mylib::adder(x,y), 10);
```

EVERYTHING is "as code":



Just run cargo doc




```
…/temp-rust  trunk  v1.93.0 ?
(ins) cargo doc -q

…/temp-rust  trunk  v1.93.0 ?
(ins)
```

ux: client

```rust
/// Adder: Add two numbers.
///
/// ```
/// let x = 5;
/// let y = 5;
/// assert_eq!(mylib::adder(x,y), 10);
/// ```
pub fn adder<T>(x: T, y: T) -> T
where T: std::ops::Add<Output = T> {
    x + y
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_adder_ok() {
        assert_eq!(adder(1, 2), 3);
    }

    #[test]
    fn test_adder_ko() {
        assert_eq!(adder(1, 2), 4);
    }
}
```

Oh, and `cargo test` checks the doc examples *by default* 😎

```
running 1 test
.
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

running 1 test
.
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

all doctests ran in 0.15s; merged doctests compilation took 0.14s
```

```
37 /// Adder: Add two numbers.
36 /// ```
35 /// let x = 5;
34 /// let y = 5;
33 /// assert_eq!(mylib::adder(x,y), 10);
32 /// ```
31 pub fn adder<T>(x: T, y: T) -> T
30 where T: std::ops::Add<Output = T> {
29     x + y
28 }
27
26
25
24
23
22
```

Your docs don't just describe your invariants, they check them! 💪

```
running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s


running 1 test
src/lib.rs - adder (line 2) ... FAILED

failures:

---- src/lib.rs - adder (line 2) stdout ----
Test executable failed (exit status: 101).

stderr:

thread 'main' (196168) panicked at /tmp/rustdoctestQGu8L7/doctest_bundle_2024.rs:8:1:
assertion `left == right` failed
  left: 10
 right: 11
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace



failures:
    src/lib.rs - adder (line 2)

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.0
0s
```

```
/// Adder: Add two numbers.
///
/// ```
/// let x = 5;
/// let y = 5;
/// assert_eq!(mylib::adder(x,y), 11);
/// ```
pub fn adder<T>(x: T, y: T) -> T
where T: std::ops::Add<Output = T> {
    x + y
}
```

DEVOXX MOROCCO

Integration tests?

Easy! Put them in `/tests` (with setup code if needed)

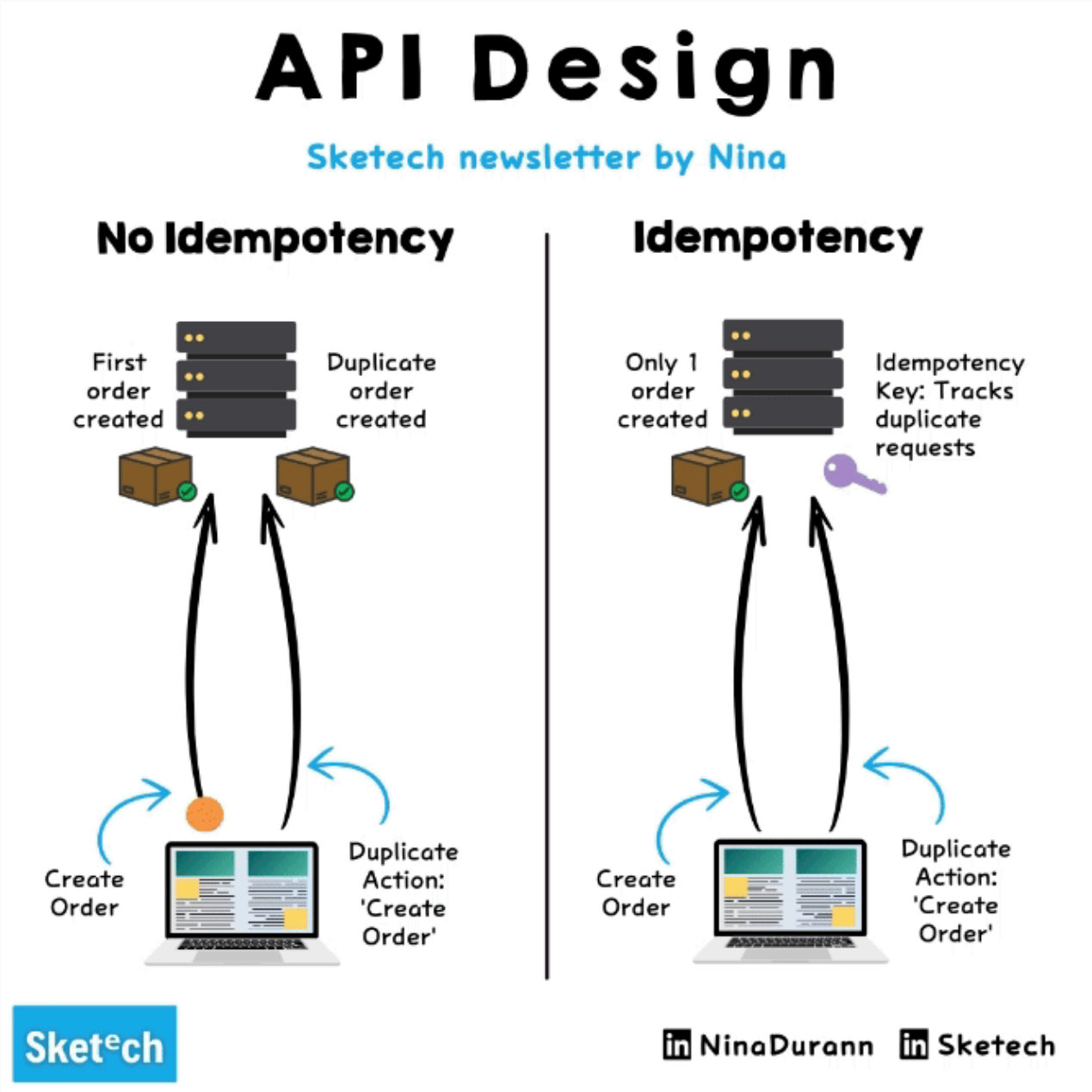
```rust
#[tokio::test]
async fn you_must_be_logged_in_to_access_the_admin_dashboard() {
    // Arrange
    let app: TestApp = spawn_app().await;

    // Act
    let response: Response = app.get_admin_dashboard().await;

    // Assert
    assert_is_redirect_to(&response, location: "/login");
}
```

```
NORMAL  trunk!  tests/api/login.rs  rust-analyzer  rust  utf-8[unix]  83% ln :49/59≡%:1
```

DEVOXX MOROCCO

By the way… We need to talk about Wiremocks 🤩

A tricky thing to test: idempotency



DEVOXX MOROCCO

The result, with Wiremocks:

```rust
#[tokio::test]
async fn newsletter_creation_is_idempotent() {
    // Arrange
    let app: TestApp = spawn_app().await;
    create_confirmed_subscriber(&app).await;

    Mock::given(matcher: path("/email")) MockBuilder
        .and(matcher: method("POST")) MockBuilder
        .respond_with(responder: ResponseTemplate::new(200)) Mock
        // Assert made at Drop
        .expect(1) Mock
        .mount(&app.email_server) impl Future<Output = ()>
        .await;

    // Act - Part 1 - Login
    app.test_user.login(&app).await;

    // Act - Part 2 - Post new issue
+-- 16 lines: let id = Uuid::new_v4().to_string();·····

    // Act - Part 3 - Post duplicate request
+--  9 lines: assert_is_redirect_to(·····

    // Assert made at Drop (See Mock)
} async fn newsletter_creation_is_idempotent
```

DEVOXX MOROCCO

This is when I decided to become a Rustacean 🦀

A complete ecosystem

We need to talk about sqlx

```rust
#[tracing::instrument(name = "Retrieve confirmed subscribers from the database", skip(pool))]
pub async fn get_confirmed_subscribers(
    pool: &PgPool,
) -> Result<Vec<Result<ConfirmedSubscriber, anyhow::Error>>, anyhow::Error> {
    let confirmed_subscribers = sqlx::query!(
        r#"
        SELECT email from subscriptions WHERE statsu = 'confirmed';
        "#,
    type annotations needed
    cannot infer type (rustc E0282)
    ------------------------------------------------------------------
    https://doc.rust-lang.org/error-index.html#E0282
    ------------------------------------------------------------------
    error returned from database: column "statsu" does not exist

    Related information:

      * mod.rs#328,9: Actual error occurred here

     (rustc)
}
```

It can handle really complex queries too!

```rust
#[tracing::instrument(name = "Retrieve presentations from the database", skip(pool))]
pub async fn get_presentations(
    pool: &PgPool,
    product_id: Uuid,
) -> Result<Vec<ProductPresentationSimple>, anyhow::Error> {
    let presentations: Vec<ProductPresentationSimple> = sqlx::query!(
        r#"
        SELECT Q.id, Q.unit_name, Q.unit_size, Q.packaging_name, Q.packaging_factor,
            Q.alternative_unit_name, Q.alternative_unit_factor, Q.created_at, Q.updat
        FROM products_presentations Q
        LEFT OUTER JOIN products_presentations_crosstable X
            ON X.presentation_id = Q.id
        LEFT OUTER JOIN products P
            ON P.id = X.product_id
        WHERE P.id = $1;
        "#,
        product_id
    )
    .fetch_all(pool)
    .await
    .unwrap()
    .into_iter()
    .map(|r| ProductPresentationSimple {
        id: r.id,
        unit_name: r.unit_name,
        unit_size: r.unit_size as u16,
        packaging_name: r.packaging_name,
        packaging_factor: r.packaging_factor as u16,
        alternative_unit_name: r.alternative_unit_name,
        alternative_unit_factor: r.alternative_unit_factor as u16,
        created_at: r.created_at,
        updated_at: r.updated_at,
    })
    .collect();

    Ok(presentations)
} async fn get_presentations
```

DEVOXX MOROCCO

The other heavyweights:

- serde/deser (serialization/deserialization)
- tera (HTML templates)
- chrono/jiff (time!)
- and many, many more... 🤩

Let's go one level higher...

Rust high-level frameworks

Axum

```rust
use axum::{ routing::{get, post}, http::StatusCode, Json, Router, };
use serde::{Deserialize, Serialize};

#[tokio::main]
async fn main() {
    tracing_subscriber::fmt::init(); // initialize tracing
    let app = Router::new()
        .route("/users", post(create_user)); // `POST /users` goes to `create_user`
    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
    axum::serve(listener, app).await;
}

#[derive(Deserialize)]
struct CreateUser { username: String, }

#[derive(Serialize)]
struct User { id: u64, username: String, }

async fn create_user(
    Json(payload): Json<CreateUser>, // Parse the request body as JSON into a `CreateUser` type
) -> (StatusCode, Json<User>) {
    let user = User { id: 1337, username: payload.username, };
    (StatusCode::CREATED, Json(user)) // Converts into a JSON response with 201 HTTP code
}
```

Actix-web

JSON

`Json<T>` allows deserialization of a request body into a struct. To extract typed information from a request's body, the type `T` must implement `serde::Deserialize`.

```
use actix_web::{post, web, App, HttpServer, Result};
use serde::Deserialize;

#[derive(Deserialize)]
struct Info {
   username: String,
}

/// deserialize `Info` from request's body
#[post("/submit")]
async fn submit(info: web::Json<Info>) -> Result<String> {
   Ok(format!("Welcome {}!", info.username))
}
```

And the Rustacean world moves fast. Really fast… 🏃💨

Dioxus: one framework, client and server!

Introduction

Dioxus is a framework for building cross-platform apps with the Rust programming language. With one codebase, you can build apps that run on web, desktop, and mobile platforms.

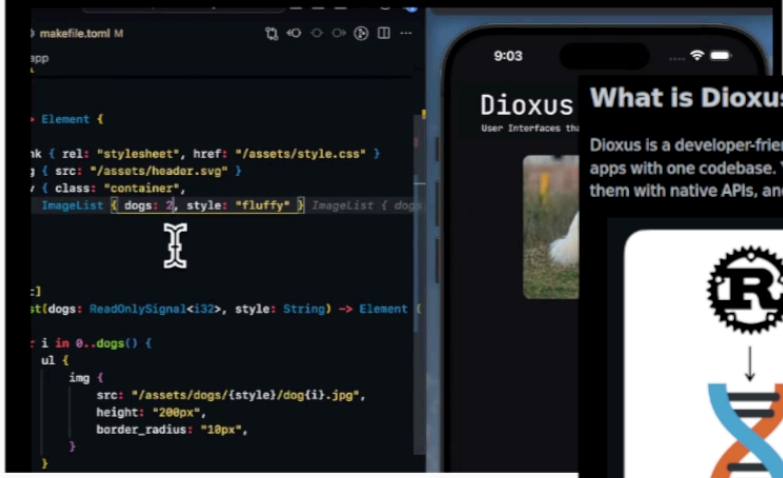
src/readme.rs

```rust
use dioxus::prelude::*;

pub fn App() -> Element {
    let mut count = use_signal(|| 0);

    rsx! {
        h1 { "High-Five counter: {count}" }
        button { onclick: move |_| count += 1, "Up high!" }
        button { onclick: move |_| count -= 1, "Down low!" }
    }
}
```

You can literally build anything with it 🤯

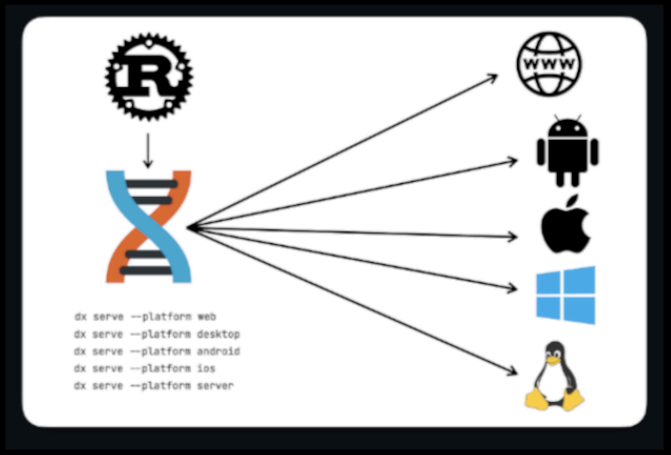
Stellar Developer Experience

With Dioxus, we try to maintain a very high bar for developer experience. We believe that building apps should be fun and straightforward. We've worked to push forward Rust itself, developing technologies like Subsecond Rust hot-reloading, WASM bundle-splitting, linker-based asset bundling, and a modular WGPU-based HTML/CSS renderer.



What is Dioxus?

Dioxus is a developer-friendly framework that empowers developers to ship cross-platform apps with one codebase. You write your apps in Rust, style them with HTML/CSS, enhance them with native APIs, and distribute them as platform-native bundles.

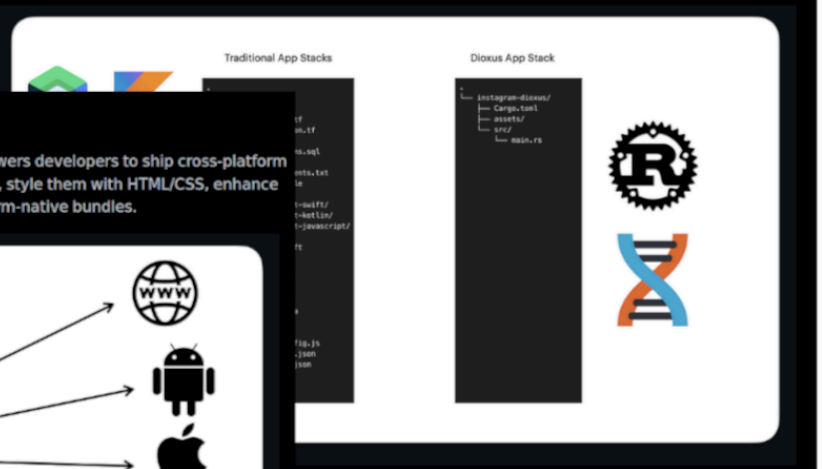


In many ways, Dioxus is similar to Flutter: we integrate our own build tools, foster an ecosystem, and provide a GUI framework. In key areas, Dioxus takes a different approach:

- Apps are declared with HTML and CSS instead of a custom styling solution
- Reactivity is inspired by web frameworks like React and SolidJS
- Dioxus code runs natively with no virtual machine, enabling zero-overhead calls to system APIs

Why Dioxus?

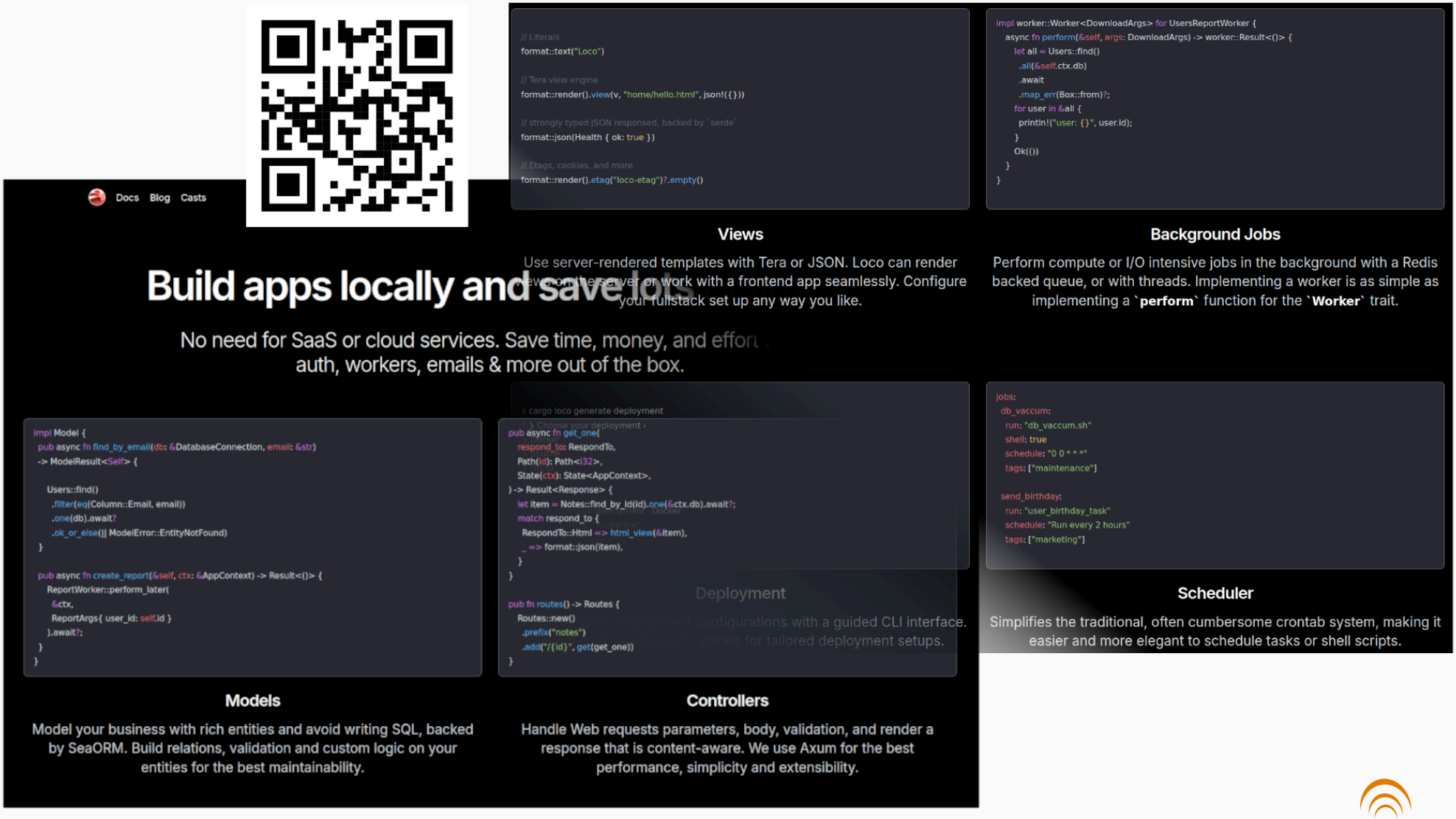
We built Dioxus because we believe the current standard of building apps is too complex. Developers need to learn and install dozens of different tools just to get their app into the world. We need a simpler and more powerful framework to bring apps from ideas to reality.



k that is fast, flexible, and has a minimal learning curve.
launch their app from idea to production as fast as
ls and a simpler architecture make it easier to develop
ship faster and are more likely to succeed.

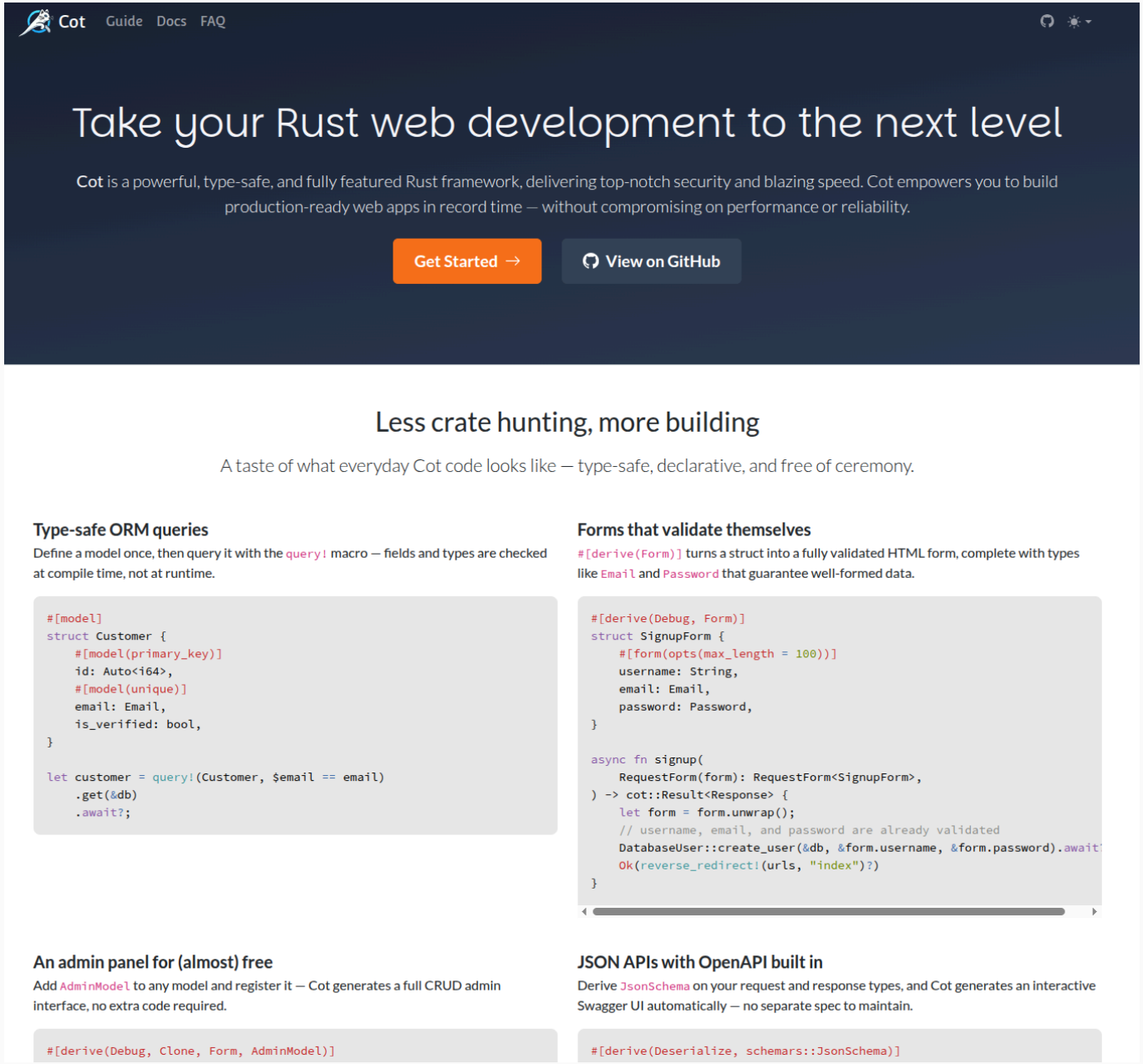
DEVOXX MOROCCO

Coming from Ruby on Rails? No problemo… there's Loco 🚂



DEVOXX MOROCCO

There's something for every taste!



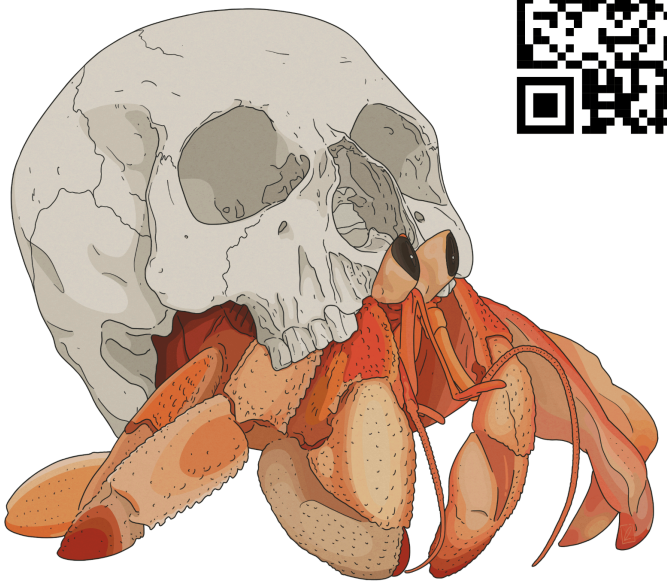
DEVOXX MOROCCO

Want to go further?

(because Rust has plenty more wild stuff like this 😅)

ZERO TO PRODUCTION IN RUST

AN OPINIONATED INTRODUCTION TO BACKEND DEVELOPMENT



LUCA PALMIERI

Learn Rust With Entirely Too Many Linked Lists

I fairly frequently get asked how to implement a linked list in Rust. The answer honestly depends on what your requirements are, and it's obviously not super easy to answer the question on the spot. As such I've decided to write this book to comprehensively answer the question once and for all.

In this series I will teach you basic and advanced Rust programming entirely by having you implement 6 linked lists. In doing so, you should learn:

- The following pointer types: `&`, `&mut`, `Box`, `Rc`, `Arc`, `*const`, `*mut`, `NonNull`(?)
- Ownership, borrowing, inherited mutability, interior mutability, Copy
- All The Keywords: struct, enum, fn, pub, impl, use, ...
- Pattern matching, generics, destructors
- Testing, installing new toolchains, using `miri`
- Unsafe Rust: raw pointers, aliasing, stacked borrows, UnsafeCell, variance

Yes, linked lists are so truly awful that you deal with all of these concepts in making them real.

Everything's in the sidebar (may be collapsed on mobile), but for quick reference, here's what we're going to be making:

1. A Bad Singly-Linked Stack
2. An Ok Singly-Linked Stack
3. A Persistent Singly-Linked Stack
4. A Bad But Safe Doubly-Linked Deque
5. An Unsafe Singly-Linked Queue
6. TODO: An Ok Unsafe Doubly-Linked Deque
7. Bonus: A Bunch of Silly Lists

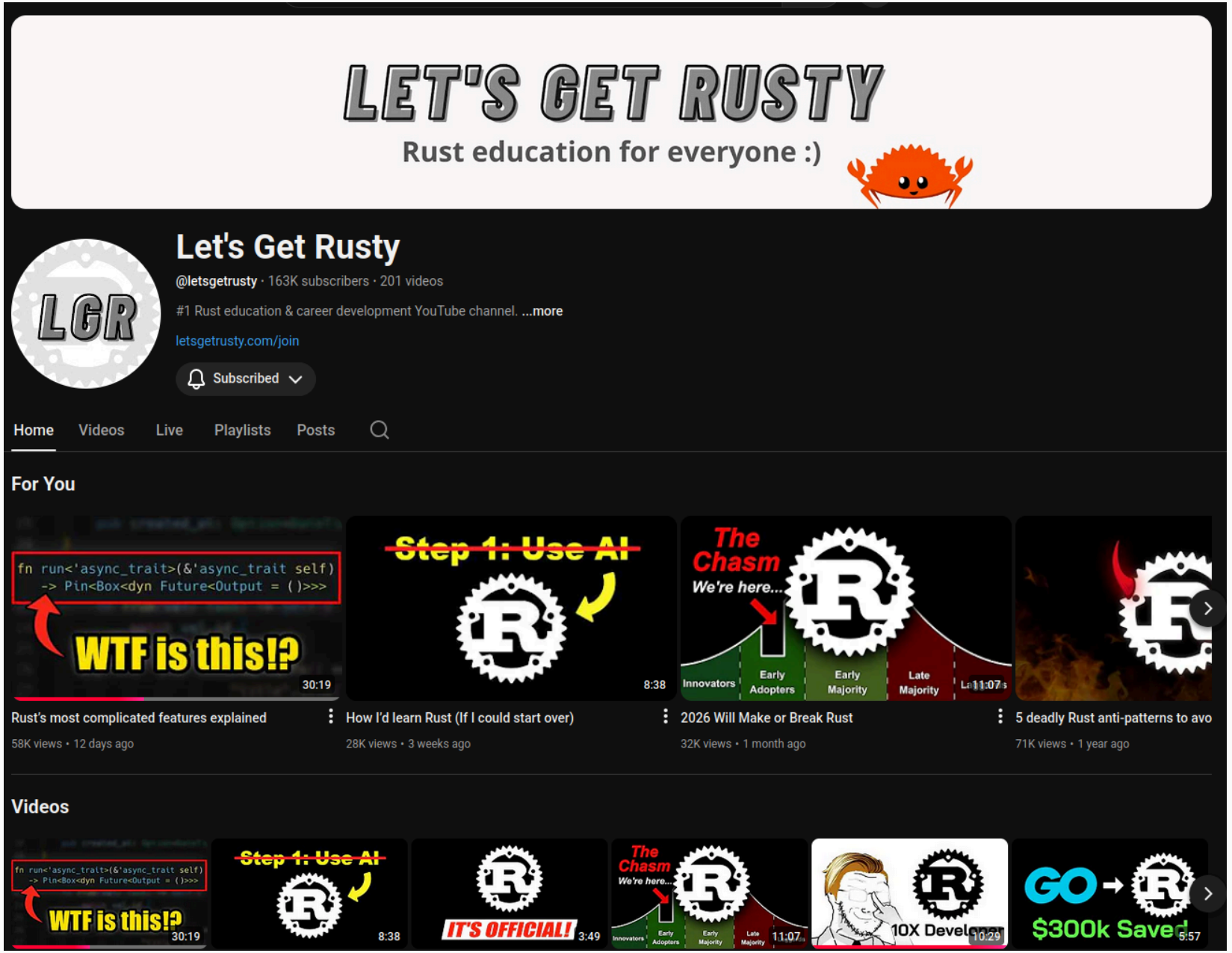
Benjamin Geer

Créer une application hypermédia en Rust

Aperçu de l'application

L'application qui sert d'exemple dans Hypermedia Systems est une application CRUD qui gère une liste de contacts. Le livre montrer comment créer une application hypermédia qui offre bonne interface utilisateur, en alternative aux applications monopages. Grâce à une minuscule bibliothèque JavaScript (htmx) et à quelques attributs déclaratifs sur les éléments HTML, toutes sortes d'interactions utilisateur peuvent récupérer des fragments de code HTML depuis le serveur et ne modifier que certaines parties de la page. Par exemple, quand on passe d'une page à une autre dans la liste des contacts ou qu'on supprime un contact, la liste affichée s'actualise sans que la page ne soit rechargée.

YouTube: Let's Get Rusty !



DEVOXX MOROCCO

Stéphane TRÉBEL

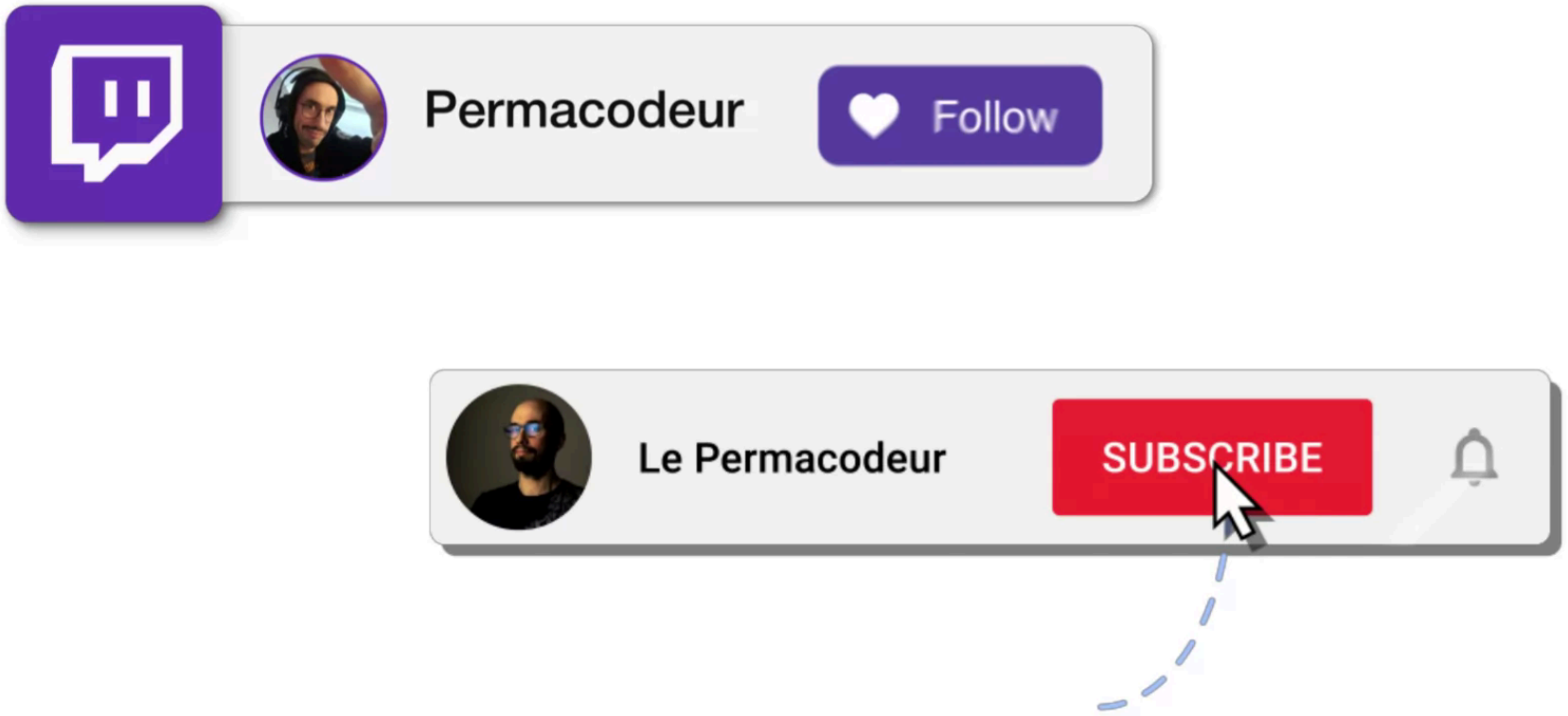
Le Permacodeur

ERP 👔

Web 🌍

Rust 🦀

DEVOXX MOROCCO


Permacodeur
Follow
Le Permacodeur
SUBSCRIBE

Permacodeur
🤩 Advent Of Code 2025...in Rust ! Jour 8/12 - Partie 1 🦀

```
fold(junction_box_list: &Vec<Coords>, connection_count: usize) -> usize {
    let mut nearest_list: BTreeMap<Coords, Coords> = BTreeMap::new();
    let mut nearest_list_sorted: HashMap<(Coords, Coords), i64> = HashMap::new();
    let max_distance: i64 = i64::MAX;

    for junction_box: &(u32, u32, u32) in junction_box_list {
        for other_box: &(u32, u32, u32) in junction_box_list {
            if junction_box < other_box {
                nearest_list_sorted.insert(
                    k: (*junction_box, *other_box),
                    v: compute_euclidian_distance(first_box: junction_box, second_box: other_box),
                );
            } else if junction_box > other_box {
                nearest_list_sorted.insert(
                    k: (*other_box, *junction_box),
                    v: compute_euclidian_distance(first_box: junction_box, second_box: other_box),
                );
            }
        }
    }
    let nearest_list_sorted = ne

    println!("Nearest list: {
    println!();
    println!("Nearest list le

    println!("Nearest list, s
    for (couple: &((u32, u32,
        println!("{distance}
    }
```

```
nearest_list_sorted HashMap<((u32, u32, u32), (u32, u32, u32)), i64, RandomState> v [LS]
nearest_list BTreeMap<(u32, u32, u32), (u32, u32, u32), Global>                    v [LS]
needs_drop()~(use std::intrinsics::needs_drop) const fn() -> bool                  f [LS]
needs_drop()~(use std::mem::needs_drop) const fn() -> bool                         f [LS]
net(use core::net)                                                                 M [LS]
net(use std::net)                                                                  M [LS]
net(use std::os::linux::net)                                                       M [LS]
net(use std::os::unix::net)                                                        M [LS]
new                                                                                [A]
```

```
INSERT  trunk!  src/main.rs[+]  rust-analyzer                rust  utf-8[unix]  36% ln:123/334≡%:33  W:2(L104)
-- INSERT --
```

```
test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 3 filtered out; finished in 0.00s

stderr ———

thread 'tests_fold::fold_sample' (1550824) panicked at src/main.rs:310:9:
assertion `left == right` failed
  left: 28
 right: 40
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

Cancelling due to test failure: 1 test still running
Cancelling [ 00:00:06] [                                        ]  3/4: 1 running, 0 passed, 0 skipped
           [ 00:00:06] aoc2025::bin/aoc2025 tests_fold::fold_gouz
```

electroradio.fm
Claptone - Clapcast 541

distances, je les trie, et pour les 1000 premières : C'est la fonction de connexion la clé où faut être sur de faire tous les cas : a et b n'ont pas de circuit, a n'a pas de circuit, b oui, b n'a pas de circuit, a oui, a et b ont un circuit (on merge toutes les boxes du circuit de b dans le circuit de a)
Alexandre_grdl: Attends, la partie 1 c'est de faire 1000 connexions,
Alexandre_grdl: Tu fais 1000 connexions ou tu traites 1000 boxes ?
Brack_0: @Alexandre_grdl yep pareil
Brack_0: j'ai juste un algo dédié pour la tambouille que tu décris Alexandre
Alexandre_grdl: Je comprends pas trop la boucle que tu fais. Parce qu'en fait, très bêtement, si je veux faire tous les couples possibles sans doublons , tu fais for i = 0 ; i < taille; i++ { for j = i+1 (très important); j < taille; j++)
Alexandre_grdl: là tu vas avoir les doublons. (a,b) et (b,a)

t _ n e a r ↓ ⏎ _ = _ n e

Cabinet
GOURDEL & TRÉBEL Software
BUREAU D'ÉTUDES & D'INGÉNIERIE LOGICIELLE

My links ✍️

Rust 🦀 now has everything you need to deliver value with your business applications. Give it a look! 👀